

Inter-Agent Systems with Strands, Agents, Amazon Bedrock, MCP, and A2A

Workshop Guide

Table of Contents

Inter-Agent Systems with Strands, Agents, Amazon Bedrock, MCP, and A2A

Prologue

- Access Your Environment
- Access CloudWatch Logs
- Understanding the MCP Gateway

Chapter 1: Building the Foundation

- Understanding Agent Communication
- Understanding the Universal Orchestrator
- Your First Request: Learning from Failure
- Deploy Your First Agent

Chapter 2: Testing and Understanding Your System

- Testing with Your First Agent
- What Happened Behind the Scenes

Chapter 3: Adding Specialized Filtering

- Deploy the Geography Agent
- Testing Multi-Agent Coordination
- Coordination Between Multiple Agents

Chapter 4: Adding Domain Expertise

- The Domain Specialist Pattern
- Deploy the Restaurant Agent
- Testing Three-Agent Coordination
- How Three Agents Coordinate

Chapter 5: Parallel Workflows

- The Context Agent Pattern
- Testing Parallel Coordination
- How Parallel Workflows Work

Chapter 6: Build a Hotel Recommender Agent

Prompt and Implement

Understanding the Filtering Logic

Deploy the Hotel Agent

Test the Pipeline

Chapter 7: Adding Intelligence with LLM Reasoning (Optional)

The Reasoning Challenge

Deploy the Budget Agent

Testing Budget Reasoning

How LLM Agents Work

The Reasoning Agent Pattern

Chapter 8: Build an Itinerary Planning Agent (Optional)

Prompt and Implement

Adapting Reasoning Patterns

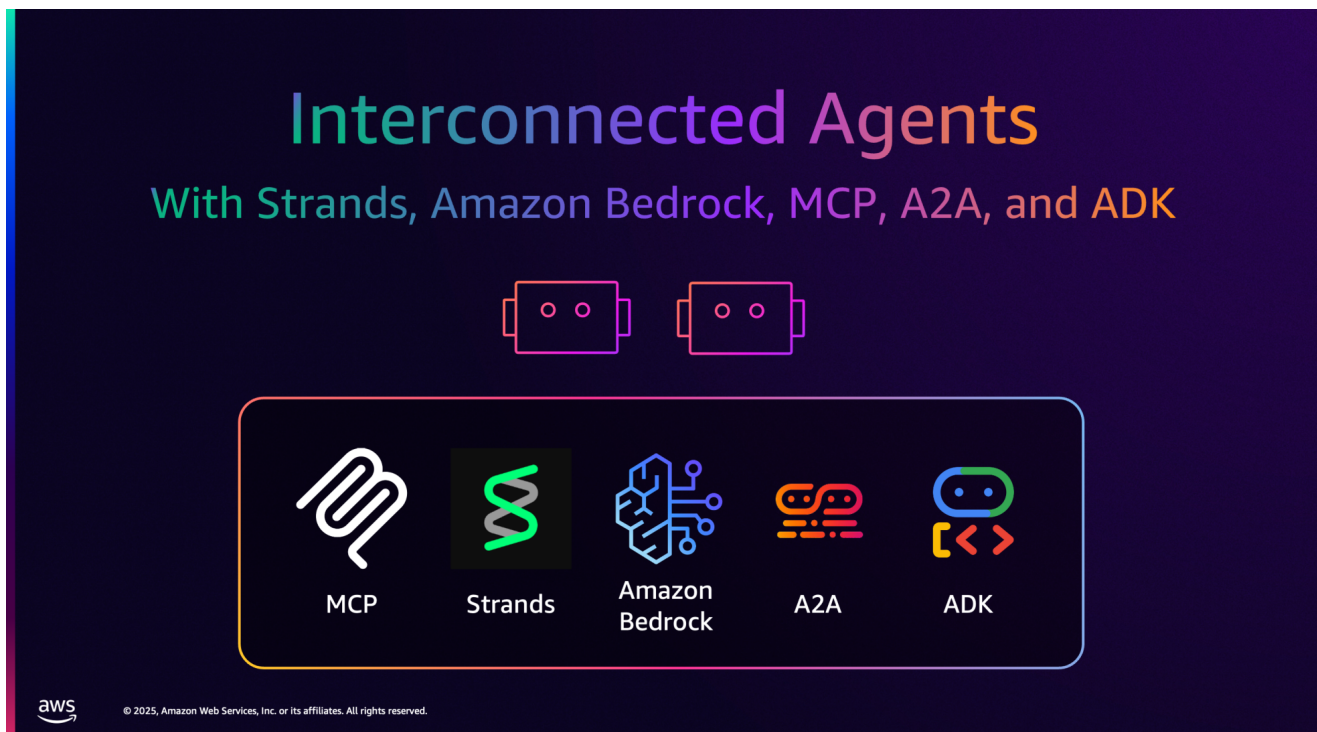
Deploy the Agent

Test Multi-Agent Coordination

Epilogue: Your Path Forward

Your Multi-Agent Journey

Cleanup



Intro

Welcome to this workshop on building, deploying, and orchestrating multi-agent systems using AWS services! In this hands-on workshop, you'll learn how to create intelligent agents that communicate, coordinate, and solve complex problems that no single agent could handle alone.

Workshop Overview

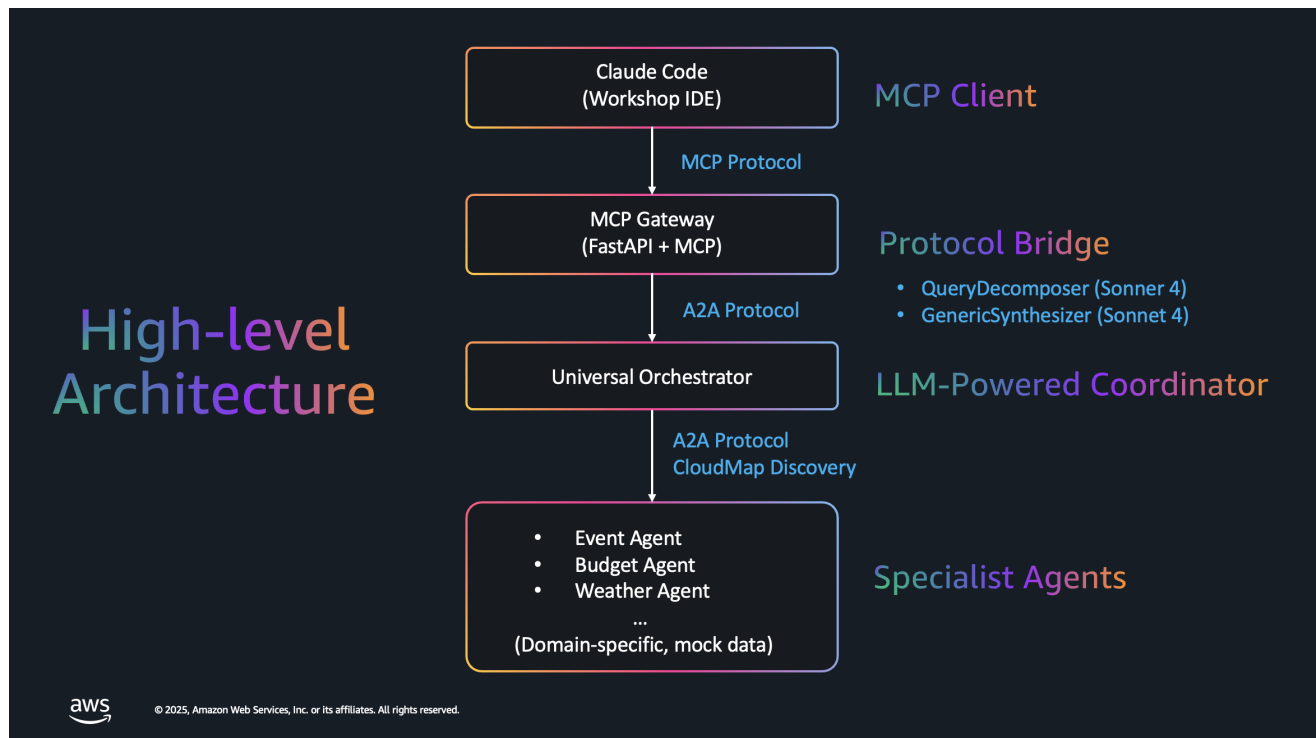
Level: Intermediate

Prerequisites: Basic AWS knowledge, Python familiarity, container concepts

By the end of this workshop, you will:

1. **Understand A2A Protocol** - Learn Agent-to-Agent communication patterns and standards
2. **Build Specialist Agents** - Create domain-specific agents with proper A2A compliance
3. **Implement LLM-Powered Orchestration** - Use Claude Sonnet 4 for intelligent agent coordination
4. **Integrate MCP Protocol** - Connect Claude Code to A2A agents via Model Context Protocol
5. **Deploy to AWS** - Build Docker images and deploy to AWS ECS with CloudMap discovery

System Architecture



High-level Architecture

Key Technologies

- **Amazon Bedrock:** Claude Sonnet 4 for LLM intelligence
- **Strands:** Agent framework with Bedrock model integration
- **A2A Protocol:** Agent-to-Agent communication standard
- **MCP Protocol:** Model Context Protocol for Claude Code integration
- **AWS CloudMap:** HTTP-based service discovery
- **Amazon Elastic Container Service (ECS):** Containerized agent deployment
- **Docker:** Container packaging and distribution

Learning Path

This workshop is organized into focused modules that build upon each other:

Module	Topic
Prologue	Environment setup, CloudWatch access, and understanding the MCP Gateway
Chapter 1	Building the Foundation (A2A protocol, orchestrator, first deployment)
Chapter 2	Testing and Understanding Your System (agent testing, context window constraints)
Chapter 3	Adding Specialized Filtering (Geography Agent, multi-agent coordination)
Chapter 4	Adding Domain Expertise (Restaurant Agent, three-agent orchestration)
Chapter 5	Parallel Workflows (Weather Agent, context provider pattern)
Chapter 6	Build a Hotel Recommender Agent (hands-on agent implementation)
Chapter 7	Adding Intelligence with LLM Reasoning (Budget Agent with Claude)
Chapter 8	Build an Itinerary Planning Agent (advanced LLM-powered reasoning)
Epilogue	Your Path Forward (reflection and next steps)

AWS Resources Used

This workshop uses the following AWS services:

- Amazon Bedrock (Claude Sonnet 4)
- ECS (container compute)
- CloudMap (service discovery)
- Application Load Balancer
- ECR (container registry)
- CloudWatch (logging and monitoring)

Estimated Cost

\$2-5 USD for the 4-hour workshop duration. All resources can be cleaned up at the end to avoid ongoing charges.

Getting Started

Ready to begin? Let's start with the prologue to set up your environment and verify everything is working correctly.

Prologue

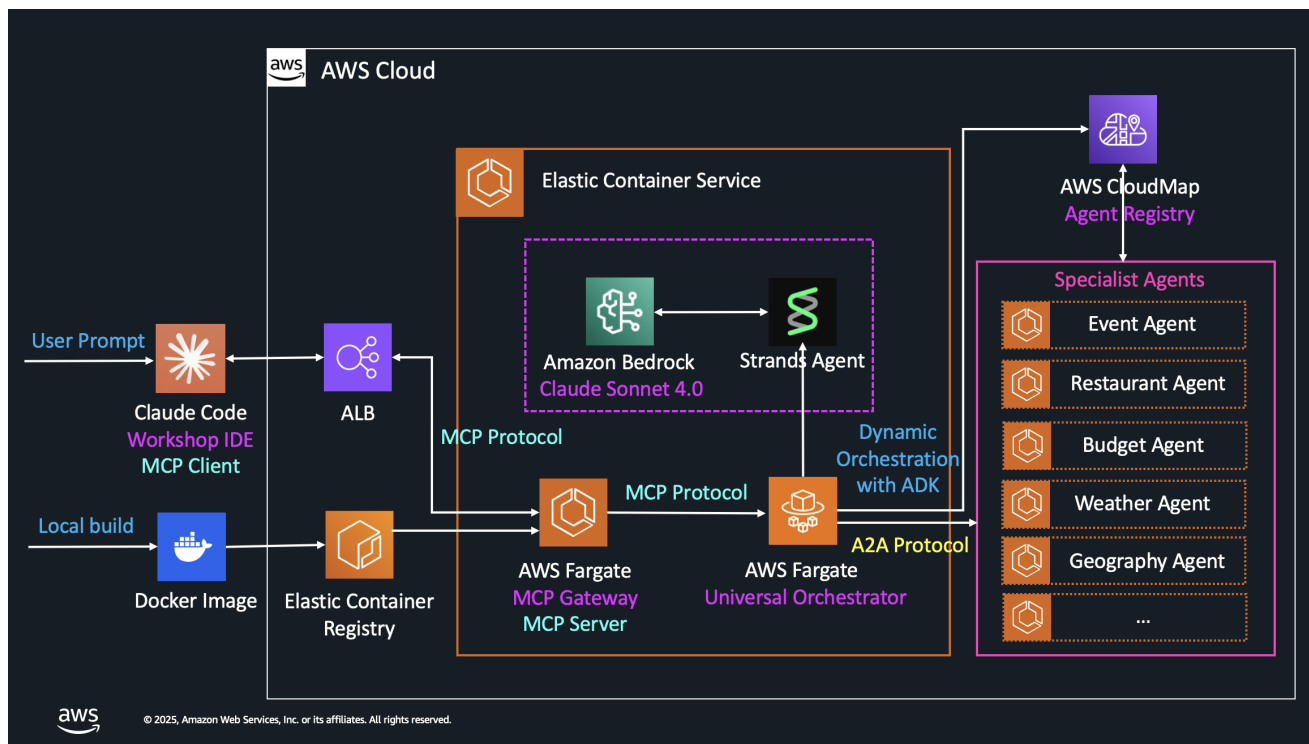
The Challenge

Imagine you're the newly appointed People Operations Coordinator at AnyCompany Inc., a 250-person SaaS company. The CEO just approved the company's first-ever all-hands offsite, bringing together 50 employees from three offices (Seattle HQ, Austin, and Boston) for a 2-day strategic planning session. The request is difficult but doable - a \$15,000 budget, tight timeline, and high visibility. The CEO wants "authentic local experience, not generic corporate conference rooms."

This workshop uses a practical business scenario to demonstrate multi-agent coordination patterns. The techniques we'll explore apply to any domain where multiple specialized services need to work together.

The Multi-Agent System

To do this, we'll build a complete multi-agent system that coordinates specialist agents using a dynamic orchestrator powered by Amazon Bedrock's Claude Sonnet 4. Here's the architecture we'll work with throughout this workshop:



Complete System Architecture

Key Components

The system consists of several intelligent components working together:

Component	Function
MCP Gateway	Translates between Claude Code (IDE assistant) and A2A agents, enabling natural language interaction with the system.
Universal Orchestrator	Coordinates agent interactions. Uses Claude Sonnet 4 to interpret queries, select relevant agents, create a workflow, and synthesize their responses.
Specialist Agents	Domain experts that work together through intelligent coordination
AWS CloudMap	Service discovery system where agents register. The orchestrator discovers agents dynamically with no hardcoded endpoints.
Amazon Bedrock	Provides the inference for our LLM intelligence (Claude Sonnet 4) that powers semantic understanding, agent selection, and response synthesis.
AWS ECS Fargate	Runs containerized agents with automatic scaling and health monitoring.

Pre-Deployed Infrastructure

The following infrastructure has already been deployed to the AWS account. There's no need to create any of this—it's ready to use.

Base Infrastructure

Service	Purpose
VPC and Networking	Foundational networking infrastructure including
ECS Fargate Cluster	ECS Cluster ready to deploy containerized agents
AWS CloudMap	HTTP namespace for agent registration and service discovery
Application Load Balancer	HTTP routing for agent services
ECR Repositories	Docker image storage for all components
IAM Roles	Permissions for Bedrock, CloudMap, EventBridge access
CloudWatch Logs	Logging and monitoring for all services
Code-Server	Browser-based VS Code environment

Agent Infrastructure

Service	Purpose
MCP Gateway	MCP Service deployed and running on ECS
Travel Orchestrator	Multi-agent coordinator deployed and running on ECS
Location-Loader	Location data service (pre-deployed, enabled in Chapter 1)
Geography Agent	Geographic filtering specialist (pre-deployed, enabled in Chapter 3)
Restaurant Agent	Dining recommendations specialist (pre-deployed, enabled in Chapter 4)
Events Agent	Event discovery specialist (pre-deployed, enabled in Chapter 4)
Weather Agent	Weather forecast specialist (pre-deployed, enabled in Chapter 5)
Budget Agent	Budget calculation specialist (pre-deployed, enabled in Chapter 7)

Workshop PDF Guide (Optional)

Take the Workshop Home

A comprehensive PDF version of all workshop content is available for download. Perfect for offline reference or if you don't complete all chapters during the workshop session.

What's included:

- All 10 chapters (Prologue through Epilogue)
- Architecture diagrams and screenshots
- Code examples and step-by-step instructions
- Agent design patterns and best practices

Download: [workshop-guide.pdf](#) (~5MB, 100+ pages)

Download Workshop Code (Optional)

You can download the code we'll be using here:

For us-east-1: [Download a2a-workshop.zip](#)

This includes all of the Agents we will be using.

Access Your Environment

Your workshop environment is a fully-configured VS Code instance running in the cloud with all the tools, code, and infrastructure you'll need. Let's get connected.

```
 :::tabs{variant="container"} ::tab{label="AWS Event"}
```

Getting Your Credentials

First, retrieve your login credentials from Workshop Studio:

1. Navigate to the **Workshop Studio** outputs tab
2. Look for these two outputs: - `a2a-workshop-main-code-server-password` - Copy this password - `a2a-workshop-main-code-server-url` - Your environment URL

Event outputs (4)					
Find outputs			< 1 >		
Key	Value	Stack name	Description	Type	
a2a-workshop-code-server-password	 vT0n[REDACTED]SB8	a2a-workshop	Code-server login password	Export	
a2a-workshop-code-server-url	 https://d3[REDACTED].cloudfront.net/?folder=/home/participant/workshop	a2a-workshop	Code-server URL for browser-based VS Code	Export	
CodeServerUser	 participant	a2a-workshop	Code-server username	Output	
MCPGatewayURL	 http://internal-a2a-mcp-gateway-alb-1402072605.us-west-2.elb.amazonaws.com/mcp	a2a-workshop	MCP Gateway URL for Claude Code configuration	Output	

Output

Security Note

Your code-server instance is secured with a unique password. The environment is isolated to your workshop session and will be automatically cleaned up after the event.

Logging In

1. Click the `a2a-workshop-main-code-server-url` link to open your environment
2. Paste the password when prompted
3. You'll be taken directly to VS Code in your browser

Welcome to code-server

Please log in below. Check the config file at `/home/participant/.config/code-server/config.yaml` for the password.

.....

SUBMIT

Login

Download Workshop Code (Optional)

If you'd like to work with the code locally or reference it later, you can download the complete workshop repository:

[Download a2a-workshop.zip](#)

:::

:::tab{label="Self-Hosting"}

Deploy the Infrastructure

To run this workshop in your own AWS account, you'll need to deploy the infrastructure using CloudFormation.

One-Click Deployment

Choose your preferred region and click the launch button:

US East (N. Virginia) - us-east-1:

Launch Stack 

US West (Oregon) - us-west-2:

Launch Stack 

Deployment Steps

1. Click the **Launch Stack** button for your preferred region
2. On the CloudFormation console: - Review the stack name (default: `a2a-workshop`) - Review the parameters (defaults are fine for the workshop) - Check the box acknowledging that CloudFormation will create IAM resources - Click **Create stack**
3. Wait for the stack creation to complete (approximately 10-15 minutes)

Getting Your Stack Outputs

Once the CloudFormation stack is complete:

1. Navigate to the **CloudFormation** console
2. Select the `a2a-workshop` stack
3. Click the **Outputs** tab
4. Look for these two outputs: - `CodeServerPassword` - Copy this password - `CodeServerURL` - Your environment URL

Security Note

Your code-server instance is secured with a unique password. The environment is isolated to your AWS account.

Logging In

1. Click the `CodeServerURL` link to open your environment
2. Paste the password when prompted
3. You'll be taken directly to VS Code in your browser

Welcome to code-server

Please log in below. Check the config file at `/home/participant/.config/code-server/config.yaml` for the password.

.....

SUBMIT

Login

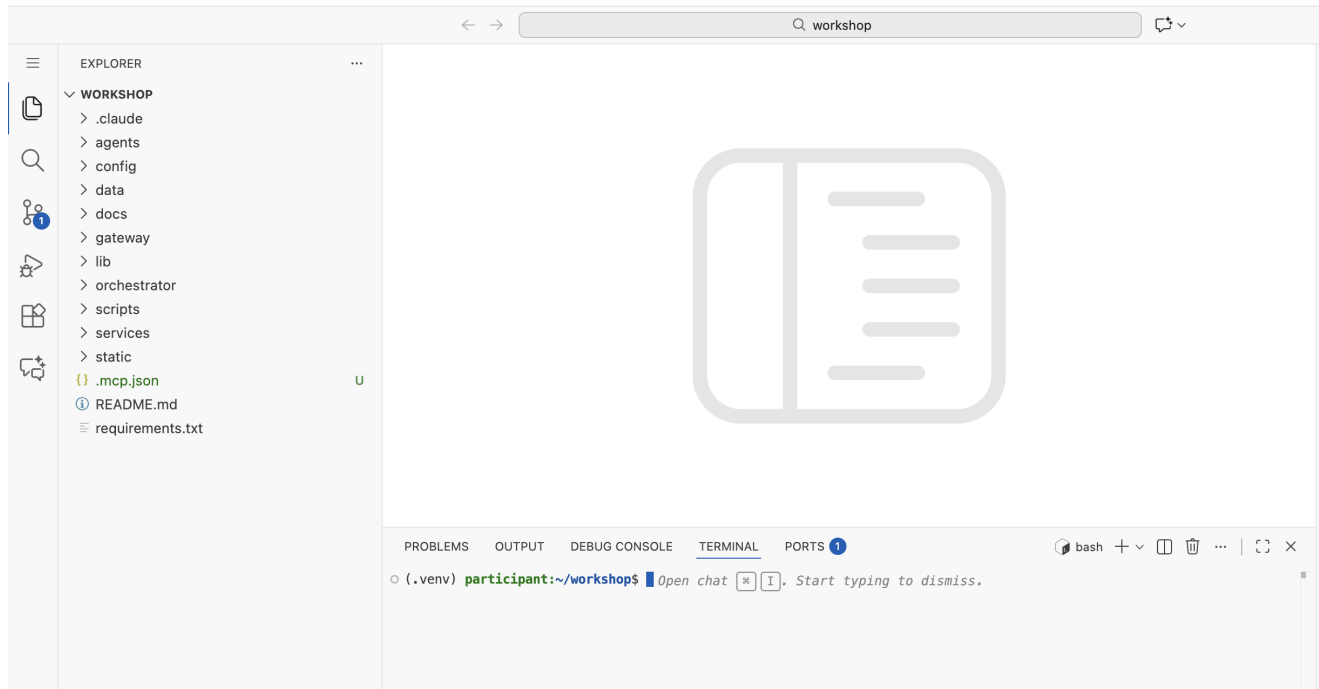
Cost Management

Remember to delete your CloudFormation stack after completing the workshop to avoid ongoing charges. Be sure to delete all of the agents and then delete the `a2a-workshop` stack from the CloudFormation console. More instructions can be found in the Epilogue.

⋮ ⋮ ⋮

Your Workshop Environment

Once logged in, you'll see a familiar VS Code interface pre-configured for the workshop:



Code Server

What's Included

- **Workshop Repository:** Complete codebase with all agents, gateway, and infrastructure code
- **Terminal Access:** Run deployment scripts, test commands, and AWS CLI operations
- **Claude Code Integration:** Pre-installed and ready to connect to your MCP gateway
- **Python Environment:** Python 3.12 with all required dependencies
- **AWS Credentials:** Pre-configured access to your workshop AWS account

You're Ready!

Your environment is fully operational. In the next step, we'll explore how the MCP Gateway works and connects Claude Code to your agent network.

Exploring the Repository

Take a moment to familiarize yourself with the repository structure:

```
WORKSHOP/
├── .claude/           # Claude Code configuration
├── agents/           # Specialized agent implementations
├── config/           # Configuration files
├── data/             # Mock data for agents
├── gateway/          # MCP Gateway code
├── lib/              # Shared libraries and base classes
├── orchestrator/     # Universal Orchestrator implementation
├── scripts/          # Deployment and utility scripts
├── services/         # Dockerfiles and service configurations
├── static/           # CloudFormation templates
├── .mcp.json         # MCP server configuration
├── README.md         # Project documentation
└── requirements.txt  # Python dependencies
```

You'll work with these directories throughout the workshop as you deploy, test, and enhance the multi-agent system.

Access CloudWatch Logs

Throughout this workshop, you'll use AWS CloudWatch Logs to observe your multi-agent system in action. CloudWatch captures detailed logs from every component - the orchestrator, MCP gateway, and all deployed agents. Understanding how to access and navigate these logs is essential for debugging, understanding agent coordination, and seeing the A2A protocol in action.

Step 1: Access the AWS Console

From your Workshop Studio event page, open the AWS Console:

1. Look for the **AWS account access** sidebar on the right
2. Click **Open AWS console**

Inter-Agent Systems with Strands Agents, Amazon Bedrock, MCP, and A2A event

- Inter-Agent Systems with Strands, Agents, Amazon Bedrock, MCP, and A2A
- Prologue
- Chapter 1: Building the Foundation
- Chapter 2: Testing and Understanding Your System
- Chapter 3: Adding Specialized Filtering
- Chapter 4: Adding Domain Expertise
- Chapter 5: Parallel Workflows
- Chapter 6: Build a Hotel Recommender Agent
- Chapter 7: Adding Intelligence with LLM Reasoning
- Chapter 8: Build an Itinerary Planning Agent
- Epilogue: Your Path Forward

AWS account access

- Open AWS console (us-west-2)
- Get AWS CLI credentials

- Workshop catalog in AWS Builder Center
- Exit event

Event ends in 2 days 23 hours 10 minutes.

Event dashboard > Inter-Agent Systems with Strands, Agents, Amazon Bedrock, MCP, and A2A

Inter-Agent Systems with Strands Agents, Amazon Bedrock, MCP, and A2A event

Event information

Start time
11/25/2025 11:22 AM

Duration
72 hours

Accessible Regions
us-west-2, us-east-1

Description
Test event for content Inter-Agent Systems with Strands Agents, Amazon Bedrock, MCP, and A2A

Workshop

Title
Inter-Agent Systems with Strands Agents, Amazon Bedrock, MCP, and A2A

Complexity level
300

Topics
Generative AI, Machine Learning (ML/AI)

AWS services
Amazon Bedrock, Amazon Elastic Container Service (Amazon ECS)

Description
In this workshop you will build AI Agents in Python using Amazon Bedrock & the open source Strands Agents framework, connect these agents to data using MCP, and connect multiple ag...
[Show more](#)

Get started

Event outputs (4)

Find outputs

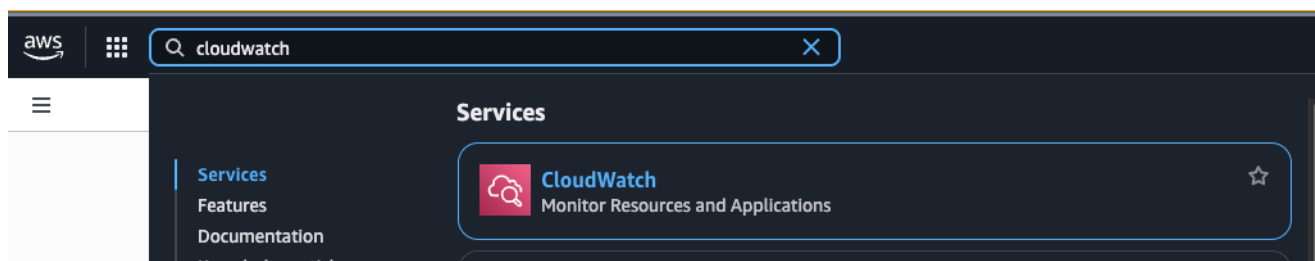
Key	Value	Stack name	Description	Type
a2a-workshop-code-server-password	M██████████cz	a2a-workshop	Code-server login password	Export
a2a-workshop-code-server-url	https://██████████.cloudfront.net/?folder=/home/participant/workshop	a2a-workshop	Code-server URL for browser-based VS Code	Export
CodeServerUser	participant	a2a-workshop	Code-server username	Output
MCPGatewayURL	http://internal-a2a-mcp-gateway-alb-644116824.us-west-2.elb.amazonaws.com/mcp	a2a-workshop	MCP Gateway URL for Claude Code configuration	Output

Access AWS Console

Step 2: Navigate to CloudWatch

Once the AWS Console opens:

1. Click the **search bar** at the top
2. Type **cloudwatch**
3. Click **CloudWatch** under Services



Navigate to CloudWatch

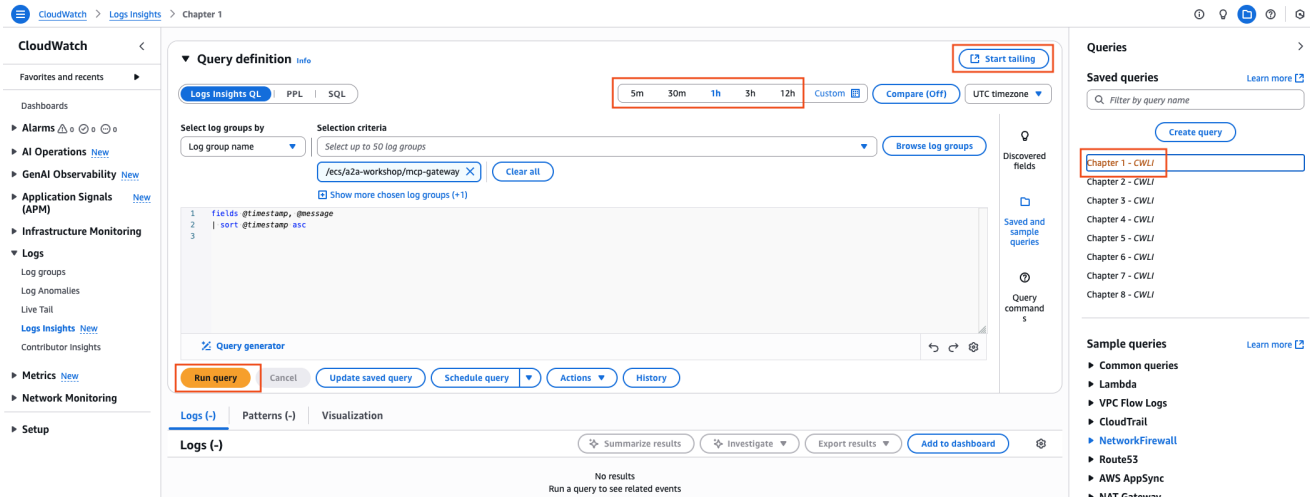
Step 3: Open Logs Insights

From the CloudWatch console:

1. In the left sidebar, expand **Logs**

2. Click **Logs Insights**

You'll see the Logs Insights query interface with pre-configured saved queries for each workshop chapter.



CloudWatch Logs Insights

Understanding Saved Queries

Your workshop includes **8 pre-configured saved queries** - one for each chapter. These queries are visible in the **Saved queries** panel on the right side of the interface, labeled "Chapter 1 - CWLI" through "Chapter 8 - CWLI".

Each query includes the log groups for all agents deployed up to that chapter:

Query	Log Groups Included
Chapter 1	mcp-gateway, orchestrator
Chapter 2	+ location-loader
Chapter 3	+ geography-agent
Chapter 4	+ restaurant-agent
Chapter 5	+ weather-agent
Chapter 6	+ hotel-agent
Chapter 7	+ budget-agent
Chapter 8	+ itinerary-agent

Progressive Queries

As you deploy more agents throughout the workshop, select the appropriate chapter query to see logs from all active components. The queries automatically include all relevant log groups for that stage of the workshop.

Using the Interface

Time Range Selector

The time range buttons control how far back to search for log entries. As you progress through the workshop, you can modify this to see only the most recent logs.

Running a Query

1. Select a saved query from the **Saved queries** panel (e.g., "Chapter 1 - CWLI")
2. Choose your **time range** (start with 5m)
3. Click the orange **Run query** button
4. View results in the **Logs** tab below

The default query sorts logs chronologically:

```
fields @timestamp, @message  
| sort @timestamp asc
```

Live Tail (Real-Time Streaming)

For real-time log monitoring while deployments run or tests execute:

1. Click the **Start tailing** button (top right)
2. Select the log groups you want to monitor
3. Watch logs stream live as events occur
4. Click **Stop tailing** when done

Live Tail is particularly useful when:

- Waiting for agent deployments to complete
- Testing queries through Claude Code
- Debugging issues as they happen
- Understanding agent coordination in real-time

Log Groups Overview

All workshop components write logs to CloudWatch with this naming pattern:

```
/ecs/a2a-workshop/{service-name}
```

Log Group	Component	Deployed In
/ecs/a2a-workshop/mcp-gateway	MCP Gateway	Chapter 1
/ecs/a2a-workshop/orchestrator	Universal Orchestrator	Chapter 1
/ecs/a2a-workshop/location-loader	Location Loader Agent	Chapter 2
/ecs/a2a-workshop/geography-agent	Geography Agent	Chapter 3
/ecs/a2a-workshop/restaurant-agent	Restaurant Agent	Chapter 4
/ecs/a2a-workshop/events-agent	Events Agent	Chapter 4
/ecs/a2a-workshop/weather-agent	Weather Agent	Chapter 5
/ecs/a2a-workshop/hotel-agent	Hotel Agent	Chapter 6
/ecs/a2a-workshop/budget-agent	Budget Agent	Chapter 7
/ecs/a2a-workshop/itinerary-agent	Itinerary Agent	Chapter 8

Correlate across components: When debugging, look at orchestrator logs first to understand workflow generation, then check individual agent logs to see task execution.

Understanding the MCP Gateway

Now that we have accessed our VS Code server, let's understand a key component of the workshop infrastructure: the MCP Gateway. This service has been pre-deployed as part of your workshop environment and serves as the communication bridge between Claude Code and your agent network.

Understanding the Gateway

The Protocol Translation Problem

Our workshop involves two different communication protocols:

- **MCP (Model Context Protocol):** Used by Claude Code in VS Code to interact with tools and services

- **A2A (Agent to Agent Protocol):** Used by our specialized agents (geography, restaurants, events, weather) to communicate with each other

These protocols speak different languages - like trying to connect a USB device to an HDMI port. The MCP Gateway is the adapter that makes them work together.

How It Works

Here's the flow when you make a request from Claude Code:

Mermaid Diagram

The Translation in Action

Here's a simplified view of how the gateway converts your request (from `gateway/main.py:207-228`):

```
# When Claude Code makes an MCP tool call,
# the gateway builds an A2A Task structure:

task = Task(
    id=task_id,                # Unique identifier
    context_id=context_id,     # Conversation context
    status=TaskStatus(
        state=TaskState.submitted, # Initial state
        timestamp=datetime.now(UTC).isoformat() + "Z",
    ),
    history=[message],         # Conversation history
    input={"query": user_message}, # Your natural language query
    artifacts=[],               # Will hold agent responses
    kind="task",
)

# Then wraps it in A2A's JSON-RPC protocol format:
a2a_message = {
    "jsonrpc": "2.0",
    "id": task_id,
    "method": "message/send",
    "params": {"message": task.model_dump(mode="json")},
}
```


The gateway handles all the complexity of protocol translation, UUID management, and response formatting so you can focus on natural language interactions with the agent network.

Gateway Infrastructure

The MCP Gateway is deployed as an ECS service with the following components: -

****Container****: Python 3.12 with FastAPI + MCP SDK - ****Load Balancer****: Internal Application

Load Balancer for VPC access - ****Storage****: ECR repository for the gateway Docker image -

****Monitoring****: CloudWatch Logs for request tracing The gateway is available at the URL

provided in your CloudFormation stack outputs, ready for Claude Code to connect.

Chapter 1: Building the Foundation

The Foundation

The MCP Gateway is deployed, which means Claude Code can now talk to your agent network. There's just one problem: you don't actually have any agents yet. The gateway is like a telephone exchange with no phones connected.

In this chapter, you'll build the foundation of your multi-agent system by exploring two critical components: the pre-deployed Universal Orchestrator and deploying your first specialized agent. Along the way, you'll learn how agents communicate, how they discover each other, and why coordination matters when working with multiple specialized intelligences.

The Seattle Challenge

Remember your assignment: plan a corporate offsite for 50 AnyCompany employees in Seattle. You need authentic local restaurants, team-building activities, and venues that work within a \$15,000 budget. This isn't the kind of question a single agent can answer well - it requires coordinating multiple sources of specialized knowledge.

To build a system that can help with this, you'll need a coordinator (the orchestrator) and at least one source of data (your first agent). This chapter gets both of those pieces in place.

What You'll Build

You'll explore and deploy one critical component in this chapter. The Universal Orchestrator is already deployed as part of your infrastructure. The orchestrator is responsible for discovering available agents, generating workflows that coordinate their work, and synthesizing their responses into coherent answers.

You'll deploy the Location Loader agent. This agent has a single job: load raw data about Seattle's restaurants, events, and venues. It might seem simple, but this simplicity is intentional - it demonstrates an important principle about how to design agents that work well together.

Ready to Start

Let's begin by understanding how agents communicate with each other, then we'll explore your orchestrator and deploy your first agent. Once your foundation is built, Chapter 2 will show you how it performs.

Understanding Agent Communication

Agents communicate using the A2A (Agent-to-Agent) protocol, built around three core concepts:

Task: Represents a unit of work containing the query, status (submitted/running/completed), conversation history, and results. The Task travels between agents, accumulating artifacts and messages.

Message: A single communication within a Task with a role (user/assistant/agent) and content. Messages build the conversation history so agents see full context.

Artifact: Contains an agent's work results - text summaries, structured data (JSON), or both. Agents package their output into Artifacts with metadata about what was done and what the results mean.

How a Request Flows

Here's what happens when you ask a question through Claude Code:

Mermaid Diagram

The Task travels through this flow, accumulating artifacts and messages. By the time it returns, it contains a complete record of what happened.

The MCP Gateway's Role

The pre-deployed MCP Gateway translates between MCP protocol (Claude Code) and A2A protocol (your agents). When you send a query through Claude Code, the gateway converts it to an A2A Task. Results get converted back to MCP format. This happens automatically - you just ask questions in natural language.

Understanding the Universal Orchestrator

The Universal Orchestrator is the brain of your multi-agent system. While individual agents provide specialized capabilities, the orchestrator is what makes them work together intelligently. It discovers what agents are available, figures out the best way to coordinate their work, and synthesizes their responses into coherent answers. This component has been pre-deployed as part of your workshop infrastructure. Let's explore what makes it so important.

What Makes an Orchestrator Necessary

Imagine you're planning that Seattle offsite and you have access to several experts: one who knows about all the restaurants in the city, another who understands geography and distances, and a third who tracks weather patterns. Each expert is valuable, but you need someone to coordinate them - someone who knows which experts to consult, in what order, and how to combine their advice into a coherent plan.

That's what the orchestrator does. It sits above your specialized agents and coordinates their work. When a complex question comes in, the orchestrator breaks it down, figures out which agents can help, determines the best order to call them, and synthesizes their responses into a useful answer.

Without an orchestrator, you'd need to manually figure out which agents to call, in what sequence, with what inputs. You'd need to take the output from one agent and format it correctly as input for the next agent. You'd need to handle errors, retries, and edge cases. The orchestrator handles all of this automatically.

How the Orchestrator Works

The orchestrator performs three main functions, and understanding these will help you appreciate what it brings to your system.

Discovery: Finding Available Capabilities

When a request comes in, the orchestrator's first job is to figure out what agents are available. It doesn't have a hardcoded list of agents - instead, it dynamically discovers them at runtime. This means your system can grow organically. Deploy a new agent, and the orchestrator immediately knows about it and can include it in workflows.

The discovery process is automatic. Each agent, when it starts up, registers itself in a service registry. The orchestrator queries that registry whenever it needs to know what's available. This makes the system flexible - agents can come and go, and the orchestrator adapts.

Workflow Generation: Planning the Work

Once the orchestrator knows what agents are available, it uses an LLM (Claude Sonnet 4, the same model you're using in Claude Code) to analyze your question and generate a workflow. This workflow specifies which agents to call, in what order, and what data to pass between them.

This is where the intelligence really shines. The orchestrator doesn't just blindly call every agent - it analyzes your question and determines the optimal strategy. Some workflows run agents in sequence, where one agent's output feeds into the next. Others run agents in parallel, combining their results at the end. Some workflows include conditional logic, choosing different paths based on what the data shows.

All of this happens dynamically for each unique request. The orchestrator isn't following pre-programmed scripts - it's generating the best workflow on the fly based on your specific question and the agents currently available.

Synthesis: Creating Coherent Answers

After the agents complete their work, the orchestrator collects all their responses and synthesizes them into a coherent answer. This isn't just concatenating text - the orchestrator uses an LLM to understand what each agent contributed, identify the key insights, resolve any conflicts or redundancies, and craft a natural language response that directly addresses your original question.

This synthesis step is what makes the multi-agent system feel like a single, intelligent assistant rather than a collection of separate tools.

The Intelligence Behind Orchestration

What makes this orchestrator "universal" is that it works with any agents you deploy. It doesn't have special knowledge about restaurants or geography or weather - instead, it discovers each agent's capabilities by reading their "agent card," a standardized description of what the agent can do.

When you deploy a new agent, that agent publishes a card describing its capabilities, what inputs it expects, and what outputs it provides. The orchestrator reads these cards and incorporates that knowledge into its workflow generation. This means you can add new agents to your system without modifying the orchestrator's code.

The orchestrator is also model-powered, which means it can handle natural language queries that don't neatly map to specific agent capabilities. It can interpret intent, handle ambiguity, and make intelligent decisions about how to fulfill requests even when they're phrased in unexpected ways.

Your First Request: Learning from Failure

Now that the orchestrator is ready to coordinate work, it's time to test the system with a real request about planning the Seattle offsite. Claude Code is already configured to use our MCP Gateway in the

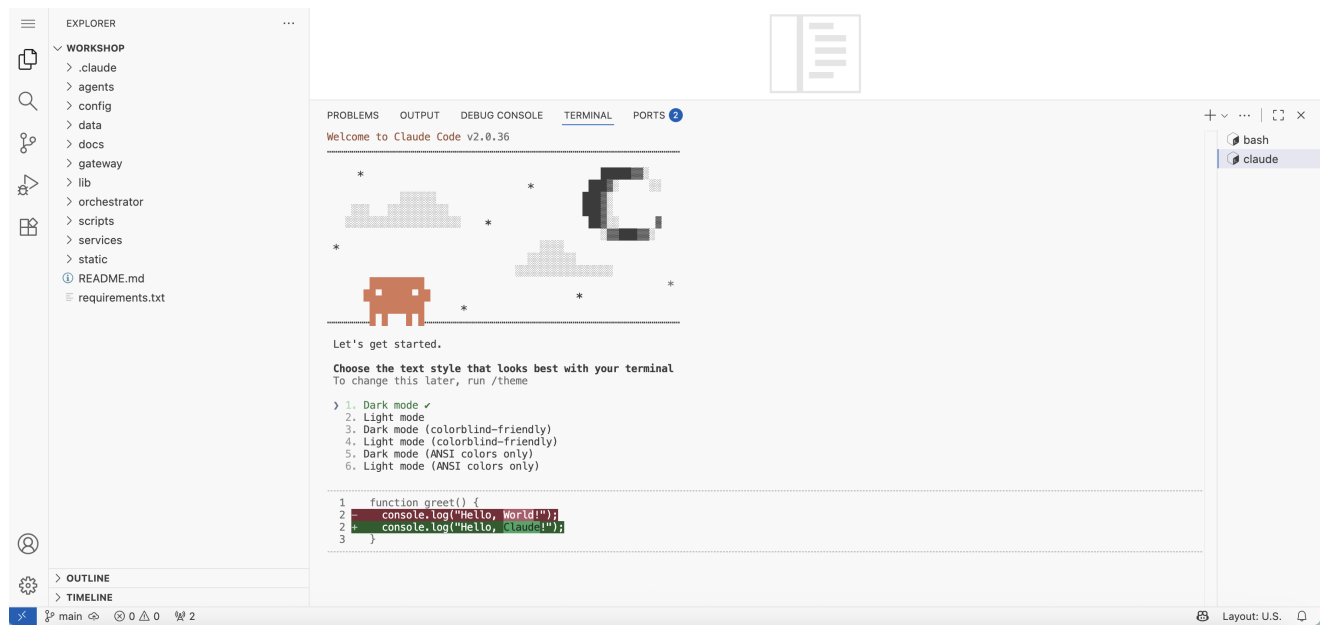
`.mcp.json` file.

Start Claude Code

Launch Claude Code by typing:

```
claude
```

You should see the Claude Code welcome screen:

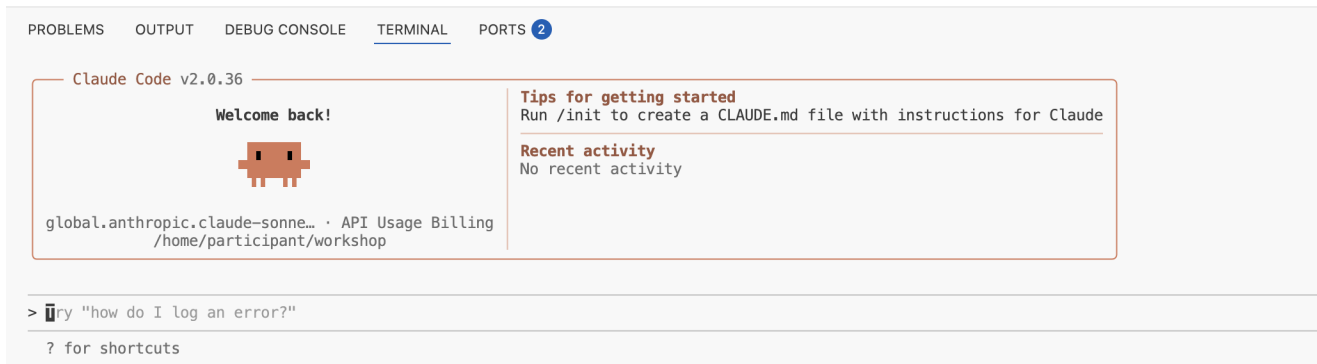


ClaudeSetup

When logging into Claude Code for the first time, you have to answer some initial setup questions. Accept all the defaults and proceed.

- Select either Light or Dark mode (your preference).

- Confirm you understand the Security Notes
- Use the recommended setting for the terminal
- Allow Claude Code to trust the files in /home/participant/workshop directory. You'll see the Claude Code interface:



ClaudeCode

You can verify that your MCP server is up and connected within Claude Code using the `/mcp` command:

```
/mcp
```

You should see something like this:

Manage MCP servers

› 1. a2a-workshop ✓ connected · Enter to view details

MCP Config locations (by scope):

- User config (available in all your projects):
- /home/participant/.claude.json
- Project config (shared via .mcp.json):
- /home/participant/workshop/.mcp.json
- Local config (private to you in this project):
- /home/participant/.claude.json [project: /home/participant/workshop]

For help configuring MCP servers, see: <https://docs.claude.com/en/docs/claude-code/mcp>

Make Your Request

Now ask Claude Code to help with the Seattle planning challenge. Notice that you're explicitly mentioning the MCP connection - this tells Claude Code to use the workshop tools you've deployed rather than just answering from its general knowledge:

```
let's use our mcp to plan a corporate offsite in Seattle for 50 people from AnyCompany. Can you suggest
```

What You'll See

Claude Code will attempt to use the MCP Gateway to fulfill your request. You'll see it making a tool call to the workshop orchestrator:

- I'll help you plan a corporate offsite in Seattle for 50 people from AnyCompany using our MCP workshop tools. Let me ask the orchestrator to coordinate the planning with appropriate specialist agents.
- a2a-workshop - ask_workshop (MCP)
 - └ Error: No agents available

This error is exactly what we want to see. The MCP Gateway successfully connected to the orchestrator, the orchestrator attempted to fulfill the request, but it discovered that no specialized agents are available yet.

Stop Claude Code from Coding

Press `Esc` in the terminal to prevent Claude from continuing.

What Claude Code Might Do Next

After seeing the error, Claude Code's behavior will vary because it's powered by an LLM. Claude loves to code and may start trying to code. We want to stop it before it does that.

Scenario 1: Troubleshooting Mode

Claude Code might try to diagnose why agents aren't available and try to create them:

- It looks like we don't have any agents available in our MCP workshop yet. Let me create a specialist agent for your Seattle recommendations.

> /workshop:create-agent is running...

- I'll help you create an event planning specialist agent for your Seattle corporate offsite! Since coordinating multiple services, I'll create a reasoning agent that can handle the sophisticated planning capabilities.
- Let me start by reading the budget agent as our template. I'm going to read an existing agent as a pattern we need to follow. For reasoning agents like event planning, I'll use the budget agent as a template for planning capabilities.
- Read(agents/budget_agent/main.py)
 | Read 796 lines

Scenario 2: Answering From General Knowledge

Alternatively, Claude Code might just answer your question using its general knowledge about Seattle instead of the workshop tools:

- It looks like the MCP workshop tool isn't currently available. Let me help you plan this Seattle corporate offsite based on my knowledge of the city!

Authentic Seattle Restaurants for Groups of 50:

- Canlis - Upscale Pacific Northwest cuisine with stunning views
- The Pink Door (Pike Place Market) - Italian with cabaret entertainment
- Ivar's Acres of Clams (Waterfront) - Seattle institution for seafood
- ...

Why We Need to Stop This

Both of these responses, while helpful in other contexts, aren't what we want for this workshop. The whole point is to learn how your deployed agent system works, not to troubleshoot it or get general knowledge answers.

In Scenario 1, Claude Code is trying to help debug the system, but you already know what's missing - you haven't deployed specialized agents yet.

In Scenario 2, Claude Code is answering from its training data about Seattle, not from your agent system. While the information might be accurate, it's not using the workshop infrastructure you've built. You want to see your MCP gateway, orchestrator, and agents working together.

Looking Under the Hood

Regardless of how Claude Code responded, something interesting happened in your system's logs during that request. Your orchestrator and MCP Gateway log everything they do to CloudWatch, which makes the system transparent and debuggable.

To see what actually happened during that request, follow the [Access CloudWatch Logs](#) guide and use the "Chapter 1 - CWLI" saved query.

Here's what those logs showed during your request:

```
# MCP Gateway receives the request
2025-11-09 05:18:55 - INFO - Using SSE streaming for ask_workshop: Plan a corporate offsite in Seattle

# Gateway sends it to the orchestrator
2025-11-09 05:18:55 - INFO - Sending A2A message to http://10.0.1.13:8000/message/send

# Orchestrator starts processing
2025-11-09 05:18:55 - INFO - 🌀 ORCHESTRATION WORKFLOW - Starting multi-agent coordination

# The critical step: agent discovery
2025-11-09 05:18:55 - INFO - 🔍 Step 1: Discovering agents via CloudMap
2025-11-09 05:18:55 - INFO - 🔍 Starting operation=agent_discovery

# The result we expected
2025-11-09 05:18:55 - ERROR - ❌ No agents discovered

# Operation completes gracefully
2025-11-09 05:18:55 - INFO - ✓ Completed operation=agent_discovery duration=0.1s
2025-11-09 05:18:55 - INFO - ✓ Completed operation=task_processing duration=0.1s task_id=...

# Gateway sends the response back
2025-11-09 05:19:15 - INFO - ✓ Extracted synthesized text (26 chars)
2025-11-09 05:19:15 - INFO - Orchestrator completed in 20001ms
```

These logs tell the complete story. The MCP Gateway received your request, converted it to A2A format, and sent it to the orchestrator. The orchestrator started its workflow and immediately went to discover available agents via CloudMap. CloudMap returned an empty list (no agents registered yet), so the orchestrator logged "❌ No agents discovered" and completed gracefully.

These logs confirm the system is working exactly as designed. The orchestrator discovered no agents and responded appropriately. This isn't a bug - it's the expected behavior when you haven't deployed any specialized agents yet.

When complete, return to your Claude Code window

The System's Current State

Right now your system looks like this: you have the MCP Gateway (which translates protocols), the Universal Orchestrator (which coordinates work), and... no specialized agents to do the actual work. It's like having a perfectly organized factory floor with a skilled foreman, but no workers.

That's about to change. In the next section, you'll deploy the Location Loader agent, and you'll immediately see how the orchestrator discovers it and incorporates it into workflows.

Deploy Your First Agent

It's time to deploy your first specialized agent. This agent, called the Location Loader, has a simple but important job: it loads raw data about Seattle's restaurants, events, and venues. While it deploys, we'll explore what it does, why it's designed the way it is, and how the orchestrator will discover and use it.

Managing Multiple Terminals

Before deploying, set up multiple terminals in VS Code to manage deployments and Claude Code sessions efficiently:

1. **Open a second terminal:** Click the  icon in the terminal panel or use ``Ctrl+Shift+`` (backtick)
2. **Organize your terminals:** - **Terminal 1** (Bash): For deployment scripts and system commands -
Terminal 2 (Claude Code): For interactive Claude Code sessions

Tips:

- Keep deployments running in the bash terminal while working in Claude Code

- You can enable multiple agents simultaneously: `./scripts/enable-agent.sh geography-agent restaurant-agent`
- Check MCP server status anytime with `/mcp` in Claude Code

Enable the Location Loader

Important: Run scripts in your **bash terminal** (not within Claude Code).

In your bash terminal, execute:

```
./scripts/enable-agent.sh location-loader
```

While You Wait

The enable process will update the CloudFormation stack to set `DesiredCount=1`, which tells ECS to start a task from the pre-built Docker image. The task will register in CloudMap once it's healthy. Once complete, the orchestrator will immediately discover this agent and can include it in workflows.

Understanding the Location Loader

The Location Loader is intentionally simple. It does exactly one thing: when you ask it for locations in a city with a particular category (like "restaurants" or "events"), it returns all of them. No filtering, no ranking, no refinement - just the complete, raw dataset.

This might seem overly simplistic. Why build an agent that just dumps all the data without any intelligence? The answer reveals an important principle about designing multi-agent systems.

The Single Responsibility Principle

In software design, the Single Responsibility Principle says that each component should do one thing well. The Location Loader embodies this principle. Its single responsibility is data loading - retrieving comprehensive datasets from the underlying data store and making them available to other parts of the system.

When you call the Location Loader with `city: "Seattle"` and `category: "restaurants"`, it returns all Seattle restaurants from the mock data - typically several thousand entries. It doesn't filter them by distance, cuisine, price, or rating. It doesn't limit the results to a manageable number. It doesn't try to figure out which ones best match your needs. It just loads and returns the complete dataset.

This design has several advantages. First, it keeps the agent simple and maintainable. Loading data is straightforward - there's no complex logic about filtering criteria or ranking algorithms. Second, it makes the agent reusable. Different queries might need different filtering approaches, and by returning the complete dataset, the Location Loader serves all of them equally well.

Third, and most importantly, it enables composition. By separating data loading from data filtering and refinement, you create the foundation for a pipeline of specialized agents. The Location Loader provides raw materials. Other agents (which you'll deploy in later chapters) can transform those raw materials in different ways for different purposes.

What This Agent Knows

The Location Loader has access to mock data that includes Seattle restaurants, events, outdoor activities, accommodations, entertainment, and shopping. This data was pre-generated for the workshop and includes realistic details: names, addresses, coordinates, categories, and metadata.

The data volumes are substantial: nearly 2,000 restaurants, over 2,500 events, almost 1,500 outdoor activities, plus hundreds of entries for accommodations, entertainment venues, and shopping locations. This gives you realistic data volumes to see how the system behaves at scale.

The data is "mock" in the sense that it was generated for this workshop, but it's based on real Seattle locations and follows realistic patterns. This makes it perfect for learning how multi-agent systems work with actual data volumes and structures.

How Agent Discovery Works

This is your first chance to see the discovery mechanism in action. When the Location Loader's ECS service starts, it registers itself in AWS CloudMap, the service registry that's part of your infrastructure. This registration includes the service name ("location-loader"), its IP address, its port, and its health status.

The orchestrator doesn't need to be told about this new agent. The next time it receives a request, it will query CloudMap for available services, find the location-loader in the results, and fetch its agent card to understand its capabilities.

The agent card is a standardized JSON document that each agent publishes at a well-known URL. It describes what the agent can do, what inputs it expects, and what outputs it provides. For the Location Loader, the card says: "I can load all locations for a city and category. Give me a city name and a category, and I'll return the complete dataset."

This self-description is key to making the system extensible. The orchestrator doesn't have hardcoded knowledge about the Location Loader. Instead, it reads the agent card and incorporates that capability into its workflow generation. This means you can deploy new agents with new capabilities, and the orchestrator will automatically learn about them and start using them.

The Complete Flow

Here's what will happen after this deployment completes:

The Location Loader ECS service starts and registers with CloudMap. The orchestrator doesn't immediately know about this - it only discovers agents when processing requests. When you make a request in the next section, the orchestrator will query CloudMap, discover the location-loader, fetch its agent card, and generate a workflow that uses it.

This lazy discovery approach means the orchestrator doesn't waste time polling for changes. It only needs to know what's available when it's actually trying to fulfill a request.

What Gets Deployed

By the time this script completes, you'll have an ECS service running the location-loader container. This service is registered in CloudMap under the name "location-loader," writes logs to `/ecs/a2a-workshop/location-loader` in CloudWatch, and exposes an API endpoint that the orchestrator can call.

The service runs continuously, ready to handle requests. When the orchestrator calls it with a city and category, it loads the appropriate data and returns it in an A2A Artifact.

Foundation Complete

Once deployment completes, you'll have your first specialized agent running. The orchestrator now has something to coordinate. Your foundation is built:

The Universal Orchestrator can discover agents via CloudMap, generate workflows dynamically based on your queries, and synthesize results from multiple agents into coherent responses.

The Location Loader provides complete datasets for Seattle's restaurants, events, and activities. It follows the single responsibility principle: load data completely and let other components handle filtering and refinement.

The MCP Gateway connects Claude Code to your agent network, translating between MCP and A2A protocols seamlessly.

Together, these components form a complete multi-agent system foundation. In Chapter 2, you'll test this system and discover important lessons about how it performs when handling real-world data volumes.

Chapter 1 Complete

Your foundation is deployed and ready. Chapter 2 will show you how it performs and what you'll learn from testing it with the Seattle planning query.

Chapter 2: Testing and Understanding Your System

From Foundation to Function

In Chapter 1, you explored the foundation of your multi-agent system: the Universal Orchestrator that coordinates work, and deployed the Location Loader that provides data. The MCP Gateway connects everything to Claude Code. Now it's time to see how this foundation performs.

This chapter is about testing and learning. You'll make the same Seattle planning request that failed in Chapter 1, and this time you'll get real recommendations. The orchestrator will discover your Location Loader agent, generate a workflow, execute it, and synthesize the results into useful suggestions.

But success reveals complexity. While the system provides good recommendations, something important happens behind the scenes that affects how those recommendations are created. The orchestrator has to make difficult choices about which data to keep and which to discard when dealing with large datasets.

What You'll Learn

This chapter teaches you two critical lessons about multi-agent systems.

First, you'll see how the orchestrator synthesizes raw data into intelligent recommendations. The Location Loader provides nearly 2,000 restaurants and over 2,500 events - complete, unfiltered datasets. The orchestrator doesn't just dump this data on you. It uses its synthesis capabilities to analyze the data, identify good options, and present organized recommendations with context and reasoning.

Second, you'll understand the constraints that affect this synthesis. Large language models have context window limits - maximum amounts of information they can process in a single request. When your data exceeds these limits, the system must use truncation strategies to fit within the available budget. This truncation works, but it's not optimal. Understanding how and why it happens will show you the value of the specialized filtering agents you'll deploy in later chapters.

Ready to Test

Your foundation is deployed and ready. Let's see how it performs and what it teaches you about building effective multi-agent systems.

Testing with Your First Agent

Your Location Loader is deployed and registered. The orchestrator now has a specialized agent it can coordinate with. Let's retry the same Seattle planning request and see what changes when you have even a single agent available.

Make the Same Request

Open Claude Code again and ask the same question you tried earlier, making sure to clear your Claude Code session first:

```
/clear
```

```
let's use our mcp to plan a corporate offsite in Seattle for 50 people from AnyCompany.  
Can you suggest some authentic local restaurants that could host our group,  
and recommend some team-building activities we could do?
```

What You'll See This Time

This time you'll get a real response with actual recommendations. Because the system uses LLMs for synthesis, the exact format will vary, but you'll see something like this:

- I'll help you plan a corporate offsite in Seattle for 50 people! Let me use the MCP workshop to get suggestions for authentic local restaurants and team-building activities.
- a2a-workshop - ask_workshop (MCP)
 - | Based on the comprehensive restaurant data for Seattle, I can recommend several excellent options for your corporate offsite of 50 people...
- Great! I've gathered comprehensive recommendations for your corporate offsite in Seattle.

Authentic Seattle Restaurants for 50 People

Top Recommendations:

- The Walrus and the Carpenter (Ballard) - Famous oyster culture, wheelchair accessible
- Anthony's Pier 66 - Upscale waterfront seafood with corporate group capabilities
- Salty's (West Seattle) - Waterfront dining with Pacific Northwest seafood
- Palace Kitchen (Belltown) - Tom Douglas restaurant with group accommodation

Alternative Options:

- Serious Pie (Ballard) - Artisanal pizza with flexible group arrangements
- Maximilien (Pike Place Market area) - Highly rated international cuisine
- The Melting Pot - Interactive fondue perfect for team bonding

Team-Building Activities

- Pike Place Market Tours - Iconic Seattle experience
- Ballard/Capitol Hill Walking Tours - Explore local neighborhoods
- Waterfront Activities - Harbor walks or boat tours near Elliott Bay
- Corporate Cooking Classes - Seattle culinary team-building sessions

Pro Tips:

- Many restaurants are clustered in Ballard, Capitol Hill, and Belltown
- Several venues offer wheelchair accessibility and outdoor seating
- Book reservations first, then plan activities in the same neighborhoods

What's Working Well

The orchestrator discovered your new agent automatically without any reconfiguration. You deployed the location-loader, and the orchestrator immediately incorporated it into its workflows. The orchestrator then analyzed your natural language question, determined which agent could help, generated an appropriate workflow, and executed it successfully. The `location-loader` agent successfully loaded the data and presented it to our Orchestrator.

What's Still Missing

While this is much better than getting an error, the recommendations have some limitations. Because the system currently only has the location-loader agent, the orchestrator can only work with the raw data that agent provides. Claude Sonnet 4 is very good at working with this data, but we refining the data before processing would improve the results.

Your System is Working

The foundation you built in Chapter 1 is now operational and providing useful results. The orchestrator discovers agents dynamically, generates workflows intelligently, and synthesizes data into actionable recommendations. In the next section, we'll examine the logs to see what the orchestrator actually did to fit nearly 2,000 restaurants and over 2,500 events into the model's context window. Understanding this process will show you why specialized filtering agents are so important.

What Happened Behind the Scenes

You got useful recommendations in the previous section, which is great! But something important happened behind the scenes that you should understand. The orchestrator had to make some difficult choices about which data to keep and which to discard. Let's examine the logs to see what actually occurred.

Looking at the Orchestrator Logs

While your request was being processed, the orchestrator was writing detailed logs about its decisions.

To view these logs, refer to the [Access CloudWatch Logs](#) guide and use the "Chapter 2 - CWLI" saved query to see all relevant component logs for this stage of the workshop.

Alternatively, you can view the specific log group at `/ecs/a2a-workshop/orchestrator` by navigating to CloudWatch > Log groups.

Here's what those logs showed during the synthesis phase of your request:

```
Attempt 1: ~233,586 estimated tokens (input too large to count)
  → Applying 25% truncation → 20,104 items remain

Attempt 2: ~175,190 estimated tokens (still too large)
  → Applying 25% truncation → 15,175 items remain

Attempt 3: 168,273 tokens ✓
  → Within budget! System prompt: 602 tokens
  → Total: 168,875 tokens ✓ Ready for synthesis
```

Understanding the Context Window Problem

Here's what happened: the Location Loader did its job perfectly. It returned the complete Seattle dataset - nearly 2,000 restaurants, over 2,500 events, and thousands of other location entries. This complete dataset represents roughly 922,000 tokens of information.

But there's a problem. The model the orchestrator uses for synthesis (Claude Sonnet 4) has a context window limit of 200,000 tokens. That's the maximum amount of information the model can process in a single request, including your question, all the agent data, the system instructions, and room for the response.

How Truncation Works

The orchestrator uses an iterative truncation strategy to solve this problem. It attempts to process the data and is rejected for being too large. So we truncated 25% of the data and try again. We keep following this pattern until we are successful.

The Truncation Strategy

Here's how the data flowed through the truncation loop:

Mermaid Diagram

While the truncation is not completely random - it keeps some of the first and last items in the list - it is not intelligent about what is removed. And we can be losing a lot of data in this process. A better way to do this would be to intelligently filter the data before we attempt to process it with our LLM. Geographic filtering is a perfect example of how to do this. To determine if a restaurant is within 5 miles of your venue, you need to:

- Parse the coordinates from both locations
- Apply the Haversine distance formula
- Compare the result to your distance threshold

This is a straightforward algorithmic task. It takes milliseconds to run, uses zero tokens, and produces deterministic results. Any restaurant at 4.8 miles qualifies; any restaurant at 5.2 miles doesn't. If you rely on truncation, the LLM never sees most restaurants. It can't calculate distances to restaurants that were discarded before synthesis even began. We'll see how to do that in the next chapter.

Chapter 3: Adding Specialized Filtering

In Chapter 2, you saw your multi-agent system work successfully - the orchestrator discovered the Location Loader, generated a workflow, and synthesized useful restaurant recommendations. But you also saw a significant limitation: the system had to truncate 43% of the data to fit within the model's context window. That's nearly half the restaurants discarded based on their position in the list, not their relevance to your query.

The truncation worked, but it was inefficient. The system used 168,000 tokens to process randomly-sampled data, and the best restaurant for your needs might have been in the discarded 43%. More importantly, the truncation was arbitrary - it didn't understand that you needed restaurants "near Pike Place Market." It just counted tokens and removed data until it fit.

This chapter solves that problem by deploying your first enhancement service: the Geography Agent. This agent does one thing exceptionally well - it filters locations by proximity using coordinate-based calculations. When you ask for restaurants "near Pike Place Market," it mathematically determines which restaurants are actually close by, returning only the relevant subset. No LLM needed, no token budget consumed, no guesswork involved.

What You'll Learn

This chapter introduces the enhancement service pattern - specialized agents that transform data before it reaches synthesis. You'll see how objective, algorithmic filtering (distance calculations) happens before subjective, LLM-based selection (cuisine preferences, atmosphere, reviews).

You'll also see multi-agent coordination in action. The orchestrator doesn't just discover two agents; it understands their relationships and creates an optimal workflow automatically. It knows the Location Loader provides raw data, the Geography Agent filters that data by location, and the synthesis step should only see the filtered results. This understanding comes from metadata in the agent cards - fields like `service_type` and `provides_for` that tell the orchestrator how agents relate to each other.

By the end of this chapter, you'll have deployed the Geography Agent and seen the dramatic performance difference. Instead of truncating 43% of 1,823 restaurants, the system will intelligently filter down to around 465 nearby restaurants. Instead of consuming 168,000 tokens on arbitrary data,

it will use around 103,000 tokens on geographically relevant data - and no truncation will be needed. And instead of arbitrary sampling, every restaurant in the results will actually be near your specified location.

Deploy the Geography Agent

Enable the Geography Agent

Let's enable the Geography Agent now so it can start while you read about how it works. In your bash terminal, execute:

```
./scripts/enable-agent.sh geography-agent
```

While You Wait

The enable process will start the pre-deployed Geography Agent ECS task. Once the task is healthy, it will register in CloudMap and the orchestrator will discover it alongside the location-loader.

What the Geography Agent Does

The Geography Agent is an enhancement service with a single, focused capability: filtering locations by proximity. Given a list of locations (restaurants, events, venues) and a center point (coordinates or a landmark name), it calculates the distance from that center to each location and returns only those within a specified radius.

The distance calculation uses the haversine formula, which computes great-circle distances between two points on a sphere given their longitudes and latitudes. This is standard geographic math - the same calculation your phone uses when showing you nearby restaurants. It's fast, accurate, and deterministic. A restaurant 4.8 kilometers from Pike Place Market will always be 4.8 kilometers away, regardless of how many times you calculate it.

The Geography Agent doesn't make judgments about whether a location is "good" or "appropriate." It doesn't evaluate cuisine, atmosphere, or reviews. It just does the math and returns locations within the specified distance. This focused responsibility makes it reliable, testable, and easily composable with other agents.

How the Orchestrator Knows This Agent Exists

When you deploy the Geography Agent, several things happen that enable the orchestrator to discover and use it:

The agent registers itself in AWS CloudMap under the service name "geography-agent." This registration includes its IP address and port. The orchestrator queries CloudMap whenever it needs to know what agents are available, so this registration makes the Geography Agent discoverable immediately.

The agent hosts an agent card at a well-known URL (`/well-known/agent.json`). This standardized location means the orchestrator knows exactly where to fetch capability information for any service it discovers in CloudMap. The agent card contains structured metadata that describes what the agent does and how it relates to other agents.

The Agent Card Metadata

The Geography Agent's card includes two critical pieces of metadata that help the orchestrator understand how to use it:

```
{
  "service_type": "enhancement",
  "provides_for": ["restaurant-agent", "events-agent", "hotel-agent"]
}
```

service_type: "enhancement" tells the orchestrator this is not a primary data provider and not a domain specialist. It's a supporting service that transforms data from other agents. Enhancement services typically sit between data loaders and domain specialists in workflow pipelines.

provides_for declares which agents benefit from this enhancement service's filtering. The Geography Agent can filter restaurants, events, or accommodations - any location-based data. The orchestrator uses this information when generating workflows. If the query involves restaurants and geography, the orchestrator knows the Geography Agent should process the Location Loader's restaurant data before passing it to any restaurant-specific agents.

These metadata fields enable the orchestrator to make intelligent decisions about agent ordering and data flow. It doesn't need hardcoded rules about the Geography Agent. It reads the agent card and understands the agent's role in the system.

Dynamic Landmark Resolution

One interesting capability the Geography Agent has is dynamic landmark resolution. When you ask for restaurants "near Pike Place Market," you don't need to provide coordinates. The Geography Agent searches through all available data files (events, restaurants, accommodations) to find a location matching that name, extracts its coordinates, and uses those as the center point for filtering.

This means you can use natural landmarks in your queries - "Space Needle," "University of Washington," "Pioneer Square" - and the Geography Agent will find the appropriate coordinates automatically. This makes the system more user-friendly while keeping the filtering objective and deterministic.

What Happens Next

Once deployment completes, your system will have two specialized agents: the Location Loader (primary data provider) and the Geography Agent (enhancement service). The orchestrator can now create workflows that coordinate these agents to filter data intelligently before synthesis.

In the next section, you'll test this coordination by making a request that requires both agents. You'll see the orchestrator discover both agents, analyze their capabilities, generate a workflow that uses them in the right order, and produce results without any truncation.

Testing Multi-Agent Coordination

Making a More Specific Request

Your Geography Agent is deployed and registered. The orchestrator now has two specialized agents available: the Location Loader (primary data provider) and the Geography Agent (enhancement service). Let's test this expanded capability by making a more specific request that requires both agents.

Open Claude Code and clear your session first:

```
/clear
```

Now ask a question that requires location filtering:

```
let's use our mcp to find restaurants near Pike Place Market in Seattle.
```

This query is more specific than the one you tried in Chapter 2. You're not just asking for restaurants in Seattle - you're asking for restaurants near a particular landmark. This specificity will let you see how the orchestrator coordinates the Location Loader and Geography Agent.

What You'll See

The system will process your request and return restaurant recommendations near Pike Place Market. The exact output varies since the synthesis model generates natural language, but you'll see recommendations focused on restaurants within close proximity to Pike Place Market.

What Just Happened

When the orchestrator queried CloudMap, it found both the location-loader and the geography-agent. It fetched both agent cards and analyzed their capabilities to determine how they could work together. The orchestrator then analyzed your query ("restaurants near Pike Place Market"), identified that you need location data filtered by proximity, and generated a workflow that uses the agents in the right sequence: load first, then filter. You mentioned "Pike Place Market" without providing coordinates. The Geography Agent searched through the available data, found Pike Place Market's location, and used its coordinates as the center point for filtering. The geography agent performed its task: find locations within a range of a location. Every restaurant in the response is actually near Pike Place Market. The filtering wasn't based on position in the list or random sampling - it was based on actual geographic proximity calculated with the haversine formula.

Agent Description

Much of the instructions being provided to the Orchestrator from the Agent are contained in the description of the Agent. For example, our Geography Agent gives very clear instructions to the Orchestrator what it does and when it should be used. The Orchestrator will read this description and include it when deciding what workflow to create.

Spatial filtering agent. I filter locations by proximity to landmarks.

I apply OBJECTIVE SPATIAL FILTERING using coordinate-based distance calculation.
Use me after location loading to reduce datasets by geographic relevance.

USAGE: Use me after location-loader to filter by distance.
I do objective filtering (hard data: coordinates + haversine distance).

WORKFLOW:

1. Receive: locations array from location-loader
2. Resolve: landmark coordinates (e.g., "Pike Place Market" → lat/lon) dynamically via `\_resolve_landmark`
3. Filter: calculate distance using haversine, keep only within radius
4. Return: `filtered_locations` (spatially-filtered subset, sorted by distance)

EFFICIENCY NOTE:

Filter 2,000 locations → 20 relevant ones (me, objective/fast),
then select best 10 (LLM, subjective/thoughtful).
This is more efficient than LLM reasoning about all 2,000.

WHY OBJECTIVE FIRST:

Distance calculation is deterministic and fast. Apply objective filters before subjective selection to

The Orchestrator's Intelligence

What's particularly impressive is that you didn't tell the system how to coordinate these agents. You didn't write a workflow that says "call location-loader, then call geography-agent, then synthesize." The orchestrator figured this out by analyzing the agent cards' capability descriptions and metadata (`service_type: "enhancement"`, `provides_for`) and your query's requirements (geographic proximity to a specific landmark). The orchestrator generated a workflow that coordinates the agents optimally: load all Seattle restaurants, filter to those near Pike Place Market, then synthesize restaurant recommendations from the filtered set. This workflow emerges from the orchestrator's understanding of agent capabilities, not from hardcoded rules.

Coordination Between Multiple Agents

Looking at the Orchestrator Logs

While your request was being processed, the orchestrator was writing detailed logs about its workflow generation decisions.

To view these logs, refer to the [Access CloudWatch Logs](#) guide and use the "Chapter 3 - CWLI" saved query to see all relevant component logs for this stage of the workshop.

Alternatively, you can view the specific log group at `/ecs/a2a-workshop/orchestrator` by navigating to CloudWatch > Log groups.

Here's what those logs showed during the workflow generation phase:

I need to analyze this query about finding restaurants near Pike Place Market and create a workflow with the available agents.

Query Analysis:

- Location: Seattle
 - Category: Restaurants
 - Geographic constraint: Near Pike Place Market
 - This is a classic load → filter pattern
- Available Agent Analysis:**
- `location-loader`: Can load all Seattle restaurants
 - `geography-agent`: Can filter by proximity to Pike Place Market
 - Perfect match for the agent-declared workflow pattern: "Load → Filter by geography"

Workflow Pattern:

Sequential execution is required because geography-agent needs the locations array from location-loader

Tool #3: generate_workflow

I've created an optimal workflow configuration for finding restaurants near Pike Place Market. This

Workflow Strategy:

- **Pattern:** Sequential (Load → Filter)
- **Stage 1:** Load all Seattle restaurants (complete dataset)
- **Stage 2:** Filter by proximity to Pike Place Market

How It Works:

1. **location-loader** loads all Seattle restaurants (~1,823 complete dataset)
2. **geography-agent** receives the restaurant data and:

- Dynamically resolves "Pike Place Market" coordinates
- Calculates distances using haversine formula
- Filters to nearby restaurants within reasonable radius
- Returns results sorted by distance from Pike Place Market

Why Sequential:

The geography-agent requires the `locations` array from location-loader's output, creating a true c

The Results

Let's look at what the Geography Agent accomplished. When you asked for restaurants near Pike Place Market, the system processed your request like this:

Location Loader returned 1,819 restaurants in Seattle - the complete, unfiltered dataset.

Geography Agent calculated the distance from Pike Place Market to each of those 1,819 restaurants using the haversine formula. It found that 465 restaurants are within reasonable proximity of the landmark. It returned only those 465 restaurants to the next stage.

Synthesis received a dataset of 465 restaurants, all verified to be near Pike Place Market. These 465 restaurants consumed 103,297 tokens - well within the 180,000 token budget. No truncation was needed.

The filtering happened algorithmically, before synthesis. The synthesis model never saw the 1,354 restaurants that are farther away because the Geography Agent excluded them mathematically. This is fundamentally different from the truncation you saw in Chapter 2.

Here's how the two-agent workflow processed your request:

Mermaid Diagram

The performance improvement is dramatic. In Chapter 2, when you asked for Seattle restaurants without geographic filtering:

- The Location Loader returned 1,819 restaurants
- Total: approximately 922,000 tokens of raw data
- After three truncation iterations: 168,273 tokens
- Data loss: 43% discarded to fit within budget

In Chapter 3, when you asked for restaurants near Pike Place Market:

- The Location Loader returned 1,819 restaurants (same as before)
- The Geography Agent filtered to 465 restaurants near the landmark
- Sent to synthesis: 103,297 tokens - NO TRUNCATION NEEDED
- Data loss: 1,354 restaurants excluded by geographic criteria (not arbitrary truncation)

What You've Built

Your system now has three components that work together seamlessly:

Location Loader provides complete, unfiltered datasets. It's a primary agent that gathers all available information without interpretation.

Geography Agent applies objective, algorithmic filtering based on coordinates and distance. It's an enhancement service that transforms data before synthesis.

Orchestrator discovers these agents via CloudMap, reads their capabilities from agent cards, generates workflows that coordinate them optimally, and synthesizes final results.

The orchestrator didn't need explicit programming to coordinate these agents. It read their metadata (`service_type: "enhancement"`, `provides_for`), analyzed your query's requirements, and generated a workflow that uses them in the right sequence with proper data flow.

Looking Ahead

You've solved the truncation problem for geographic queries. But your Seattle offsite planning task has other requirements: group capacity, cuisine preferences, budget constraints, weather considerations. Each of these could benefit from specialized filtering or domain expertise.

Chapter 4: Adding Domain Expertise

From Proximity to Preference

In Chapter 3, you deployed the Geography Agent and saw dramatic improvements. Instead of sending all 1,819 Seattle restaurants to synthesis and requiring truncation, the Geography Agent filtered down to 465 restaurants near Pike Place Market. These 465 restaurants fit comfortably within the token budget, using only 57% of available capacity. No truncation needed, no arbitrary sampling, no information loss.

But proximity alone doesn't guarantee relevance. Those 465 nearby restaurants include steakhouses, seafood restaurants, Italian trattorias, Thai noodle shops, burger joints, and pizza places. If you're planning a corporate offsite and need vegan options for team members with dietary restrictions, knowing a restaurant is "nearby" doesn't tell you if it serves food they can eat.

What You'll Learn

This chapter introduces the domain specialist pattern - agents that apply business logic and domain-specific knowledge to refine filtered data. You'll see how the Restaurant Agent works differently from the Geography Agent:

Geography Agent (enhancement service) applies objective, mathematical filtering. It doesn't understand what restaurants are or what cuisine means. It just knows coordinates and distances. It's a cross-cutting utility that works with restaurants, events, accommodations - anything with location data.

Restaurant Agent (domain specialist) understands restaurants as a concept. It knows about cuisine types, dietary restrictions, price ranges, ratings, and how these factors relate to each other. It applies business logic: "This restaurant has `dietary.vegan: true` in its metadata, so it can serve vegan customers." It works specifically with restaurant data, using restaurant-specific knowledge.

The key difference is deterministic, metadata-driven filtering. The Geography Agent uses mathematics (haversine formula). The Restaurant Agent uses structured data fields (`dietary.vegan`, `cuisine`, `price`, `rating`). Neither one is guessing or interpreting. Both are applying clear, deterministic rules to filter data precisely.

The Domain Specialist Pattern

Enable the Specialist Agents

Let's enable both the Restaurant Agent and Events Agent now. In your VS Code terminal, execute:

```
./scripts/enable-agent.sh restaurant-agent events-agent
```

While You Wait

The enable process will start both pre-deployed agent ECS tasks. Once the tasks are healthy, they will register in CloudMap. The orchestrator will discover these agents alongside location-loader and geography-agent when processing your next request. Let's keep reading while they start.

The Limitation of Geographic Filtering

Chapter 3's Geography Agent solved an important problem. It reduced 1,819 restaurants to 465 nearby options, eliminating the need for truncation and ensuring every restaurant in the filtered set was actually near Pike Place Market. This was objective, mathematical filtering at its best - calculate distances, apply the radius threshold, keep what matches.

But geographic proximity doesn't address preference. Those 465 restaurants near Pike Place Market represent dozens of different cuisine types, price points, and dietary accommodations. If you're planning a corporate offsite and several team members follow vegan diets, knowing which restaurants are nearby doesn't tell you which ones can actually serve food they'll eat.

This isn't a weakness of the Geography Agent. It's doing exactly what it should do: filtering by location. Dietary preferences are a different dimension entirely, requiring different knowledge and different filtering logic.

Why Name-Based Inference Fails

You might think you could identify vegan restaurants by their names. Some are obvious:

- **Wayward Vegan Cafe** - clearly vegan
- **Kati Vegan Thai** - explicitly advertises vegan cuisine

- **Pizza Pi Vegan Pizzeria** - the name tells you everything

But many restaurants with excellent vegan options don't advertise it in their names:

- **Basil Viet Kitchen** - serves Thai and Vietnamese food, happens to have extensive vegan offerings
- **Teapot Vegetarian House** - vegetarian doesn't necessarily mean vegan, but this restaurant does offer vegan dishes
- **Luna Park Cafe** - an American diner that serves vegan options alongside traditional fare
- **Emmy's Vege House** - despite the informal name, this is actually a fully vegan restaurant

Even worse, some names are misleading. A restaurant called "Green Garden" might sound plant-based, but it could be a seafood restaurant with a nature-themed name. "The Garden Room" could serve steaks. "Fresh Start" might be a juice bar with no actual food.

You need structured, explicit data about dietary accommodations. Guessing from names produces false positives and false negatives. This is exactly the kind of problem that requires domain-specific knowledge.

The Three-Agent Pipeline

When you request "vegan restaurants near Pike Place Market," the system needs to apply both geographic and dietary filtering. This requires coordinating three agents:

Location Loader provides the raw data: all 1,819 Seattle restaurants with their complete metadata including coordinates and dietary information.

Geography Agent applies the first filter: calculate distances from Pike Place Market, keep only restaurants within the specified radius. This reduces 1,819 restaurants to 465 nearby options.

Restaurant Agent applies the second filter: check the `dietary.vegan` field for each of the 465 nearby restaurants, keep only those where the value is `true`. This might reduce 465 restaurants to perhaps 5-7 vegan options.

The synthesis model receives 5-7 highly-relevant restaurants. Every single one meets both criteria: it's near Pike Place Market (geographic filter) AND it serves vegan food (dietary filter). The results are precise, relevant, and deterministically verified.

Deterministic Quality

Domain specialists use structured metadata to filter precisely. No guessing, no inference, no ambiguity. When a restaurant has ``dietary.vegan: true``, you can serve vegan customers there. When it's ``false``, you can't. This deterministic filtering produces reliable, high-quality results.

Deploy the Restaurant Agent

What the Restaurant Agent Does

The Restaurant Agent is a domain specialist that applies restaurant-specific knowledge to filter and refine restaurant data. Given a list of restaurants and filtering criteria (cuisine preferences, dietary restrictions, price range, minimum rating), it evaluates each restaurant against those criteria and returns only the matches.

The filtering is deterministic and metadata-driven. When you request vegan restaurants, the agent checks each restaurant's `dietary.vegan` field. If the value is `true`, the restaurant can serve vegan food and gets included. If it's `false` or absent, the restaurant can't serve vegan food and gets excluded. No inference, no guessing, just checking explicitly-set metadata fields.

The agent supports multiple filtering dimensions:

Cuisine type - Filter by specific cuisine types like "italian", "thai", "vietnamese", "mexican". The cuisine field can contain multiple values (e.g., "thai;vietnamese"), and the agent matches if any of them align with your request.

Dietary restrictions - Filter by dietary accommodations like "vegan", "vegetarian", "gluten_free", "halal", "kosher". The agent uses ALL-OF logic: if you specify multiple restrictions, a restaurant must satisfy all of them to be included.

Price range - Filter by price level (\$, \$\$, \$\$\$, \$\$\$\$) to match budget constraints.

Minimum rating - Exclude restaurants below a certain rating threshold to ensure quality.

The Restaurant Agent doesn't make subjective judgments. It doesn't interpret menus or guess about ingredients. It checks structured data fields that explicitly declare each restaurant's characteristics. This deterministic approach produces reliable, consistent results.

What Happens Next

Once deployment completes, your system will have three specialized agents: the Location Loader (primary data provider), the Geography Agent (enhancement service for geographic filtering), and the Restaurant Agent (domain specialist for restaurant-specific refinement).

The orchestrator can now create sophisticated workflows that coordinate all three agents. When you ask for vegan restaurants near a landmark, the orchestrator will discover these three agents, analyze their capabilities and relationships, and generate a three-stage workflow: load data, filter by geography, refine by dietary preferences.

In the next section, you'll test this three-agent coordination by making exactly that request. You'll see how the orchestrator chains the agents together and produces highly-relevant results where every restaurant meets multiple criteria deterministically.

Testing Three-Agent Coordination

Making a Dietary-Specific Request

Your Restaurant Agent is deployed and registered. The orchestrator now has three specialized agents available: the Location Loader (primary data provider), the Geography Agent (enhancement service for location filtering), and the Restaurant Agent (domain specialist for restaurant-specific refinement). Let's test this expanded capability by making a request that requires both geographic and dietary filtering.

Open Claude Code and clear your session first:

```
/clear
```

Now ask a question that requires dietary filtering:

```
let's use our mcp to find vegan restaurants near Pike Place Market in Seattle.
```

This query requires two distinct types of filtering. Geographic filtering (which restaurants are near Pike Place Market) and dietary filtering (which of those nearby restaurants serve vegan food). The orchestrator will need to coordinate all three agents to fulfill this request.

What You'll See

Claude Code will discover three agents, generate a three-stage workflow, and return highly-relevant vegan restaurant recommendations. Here's what the output looks like:

- 14 excellent vegan restaurants found near Pike Place Market

Closest option:

- El Borracho (0.05 miles away) - 4.2★ Mexican restaurant offering creative vegan options including plant-based tacos, burritos, and specialty cocktails. Casual atmosphere perfect for team gatherings.

Other nearby options:

- Rojo's Mexican Food (0.67 miles) - Authentic Mexican with extensive vegan menu including Beyond Meat options
- Thai On 1 (0.74 miles) - Thai cuisine with dedicated vegan section
- Menya Musashi (0.88 miles) - Ramen house with plant-based broths
- Kati Vegan Thai (entirely vegan-focused, delivery available)

All restaurants verified to have explicit vegan accommodations in their metadata (dietary.vegan: true). Distance calculations performed using haversine formula from Pike Place Market coordinates.

The orchestrator discovered three agents (location-loader, geography-agent, restaurant-agent), generated a three-stage sequential workflow, and executed it successfully. The entire process took about 19 seconds, with most time spent on workflow generation and LLM-based synthesis.

What Just Happened

This response demonstrates sophisticated three-agent coordination with deterministic filtering at each stage:

Discovery found three agents: When the orchestrator queried CloudMap, it found the location-loader, geography-agent, and restaurant-agent. It fetched all three agent cards and analyzed their capabilities and relationships to determine how they could work together.

Three-stage workflow generated: The orchestrator analyzed your query ("vegan restaurants near Pike Place Market"), identified that you need both geographic filtering (proximity) and dietary filtering (vegan), and generated a three-stage workflow: load all Seattle restaurants, filter by proximity to Pike Place Market, then refine by vegan dietary accommodations.

Deterministic filtering at each stage: The Geography Agent used the haversine formula to mathematically verify proximity. The Restaurant Agent checked the `dietary.vegan` metadata field to deterministically verify vegan accommodations. No guessing, no inference - just checking structured data fields.

Highly-relevant results: Every restaurant in the response meets both criteria with certainty. Each one is geographically near Pike Place Market (verified by distance calculation) AND serves vegan food (verified by metadata check).

The Three-Agent Pipeline in Action

The workflow that executed behind the scenes followed this pattern:

Stage 1 - Location Loader: Loaded all 1,819 Seattle restaurants from the data store, returning complete metadata including coordinates, cuisine types, dietary accommodations, and ratings.

Stage 2 - Geography Agent: Received the 1,819 restaurants and Pike Place Market as the center point. Calculated distances using haversine formula. Kept only restaurants within reasonable proximity (465 restaurants).

Stage 3 - Restaurant Agent: Received the 465 nearby restaurants. Checked the `dietary.vegan` field for each one. Kept only restaurants where the value is explicitly `true` (14 vegan restaurants).

Synthesis: Received 14 highly-relevant restaurants, all verified to be both nearby and vegan. Generated natural language recommendations with context about each option, including the closest match (El Borracho at just 0.05 miles away).

The data volume decreased at each stage through deterministic filtering: 1,819 → 465 → 14. The synthesis model worked with a small, precise dataset where every entry met your exact criteria.

Mermaid Diagram

Understanding the Agent Relationships

What's particularly impressive is how the orchestrator understood the relationships between these agents without explicit instructions. It recognized:

The Geography Agent's metadata declares `provides_for: ["restaurant-agent"]`, indicating it enhances restaurant data.

The Restaurant Agent's metadata declares `optional_enhancements: ["geography-agent"]`, indicating it works better with pre-filtered geographic data.

Based on these metadata declarations, the orchestrator reasoned: "Geography Agent should run before Restaurant Agent. Geography provides location filtering that Restaurant can use. This creates an efficient pipeline."

The workflow emerged from agent metadata and query analysis, not from hardcoded rules. You could deploy additional agents with different relationship declarations, and the orchestrator would incorporate them into workflows automatically.

Comparing to Chapters 2 and 3

The evolution of your system is striking:

Chapter 2 (Location Loader only): All 1,819 restaurants sent to synthesis, requiring three truncation iterations to fit within token budget. Random sampling based on list position. Results included restaurants all over Seattle, with no dietary filtering.

Chapter 3 (Location Loader + Geography Agent): 465 nearby restaurants sent to synthesis, no truncation needed. Results all near Pike Place Market, but mixed dietary options - some vegan, some not.

Chapter 4 (Location Loader + Geography + Restaurant): 14 highly-relevant restaurants sent to synthesis. Results meet both criteria: nearby AND vegan. Perfect precision through multi-stage deterministic filtering.

Each agent added a new filtering dimension. The results became increasingly precise, increasingly relevant, and increasingly useful.

How Three Agents Coordinate

Looking at the Orchestrator Logs

While your vegan restaurant request was being processed, the orchestrator was writing detailed logs about its three-agent workflow generation decisions.

To view these logs, refer to the [Access CloudWatch Logs](#) guide and use the "Chapter 4 - CWLI" saved query to see all relevant component logs for this stage of the workshop.

Alternatively, you can view the specific log group at `/ecs/a2a-workshop/orchestrator` by navigating to CloudWatch > Log groups.

Here's what those logs showed during the workflow generation phase:

```
I've created an optimal workflow configuration for finding vegan restaurants near Pike Place Market.
**Workflow Strategy:**

- **Pattern**: Sequential (Load → Filter → Refine)
- **Stage 1**: Load all Seattle restaurants
- **Stage 2**: Filter by proximity to Pike Place Market
- **Stage 3**: Refine to vegan options

**How It Works:**

1. **location-loader** loads all Seattle restaurants (~1,823 complete dataset)

2. **geography-agent** receives restaurant data and:

- Resolves "Pike Place Market" coordinates dynamically
- Filters to nearby restaurants using spatial calculations
- Returns proximity-filtered results sorted by distance

3. **restaurant-agent** receives nearby restaurants and:

- Applies vegan dietary restriction filtering
- Sorts results by rating (highest first)
- Returns final refined list of vegan restaurants

**Why This Pattern:**
This follows the agents' declared efficiency principle: "Load 2,000 → Filter to 20 (objective/fast)"
```

The Three-Stage Workflow Structure

Here's how the three-agent workflow processed your vegan restaurant request:

Mermaid Diagram

The generated workflow chains agents together through input_mapping. Here's a simplified view of what the workflow structure looks like:

```
{
  "nodes": [
    {
      "id": "load_restaurants",
      "type": "agent_call",
      "agent_id": "location-loader",
      "input_mapping": {
        "location": "Seattle",
        "category": "restaurants"
      }
    },
    {
      "id": "filter_by_proximity",
      "type": "agent_call",
      "agent_id": "geography-agent",
      "input_mapping": {
        "locations": "$.load_restaurants.locations",
        "center": "Pike Place Market",
        "radius_km": 1.0
      }
    },
    {
      "id": "refine_by_dietary",
      "type": "agent_call",
      "agent_id": "restaurant-agent",
      "input_mapping": {
        "locations": "$.filter_by_proximity.filtered_locations",
        "dietary_restrictions": ["vegan"]
      }
    }
  ]
}
```

Notice how each stage's input_mapping references the output from the previous stage:

The geography-agent's `locations` input comes from `$.load_restaurants.locations` - the output of the location-loader.

The restaurant-agent's `locations` input comes from `$.filter_by_proximity.filtered_locations` - the output of the geography-agent.

This chaining creates a data pipeline where each agent transforms the data and passes it to the next stage. The workflow is like an assembly line: each station (agent) performs its specialized operation and hands the work product to the next station.

Comparing Two-Agent vs Three-Agent Workflows

In Chapter 3, you saw a two-agent workflow:

```
Location Loader → Geography Agent → Synthesis
1,819 items      465 items      (synthesis receives 465)
```

In Chapter 4, you're seeing a three-agent workflow:

```
Location Loader → Geography Agent → Restaurant Agent → Synthesis
1,819 items      465 items      14 items      (synthesis receives 14)
```

The addition of the Restaurant Agent creates another filtering stage, further refining the data before synthesis. Each stage applies deterministic filtering:

- Location Loader: No filtering, returns complete dataset
- Geography Agent: Distance-based filtering (haversine formula)
- Restaurant Agent: Metadata-based filtering (`dietary.vegan == true`)

The synthesis model benefits from this multi-stage refinement. Instead of receiving 465 restaurants and trying to identify vegan options through inference, it receives 14 restaurants that are already verified to be vegan. The synthesis can focus on subjective recommendations (which vegan restaurant is best for your group) rather than objective filtering (which restaurants are vegan).

Try One More Query

Before moving to Chapter 5, let's run one more query.

```
let's use our mcp to find some fun activities in Seattle for the next few days
```

Remember this response. In Chapter 5, you'll run the same query and see what has changed.

Chapter 5: Parallel Workflows

Breaking from Sequential Patterns

Chapters 2 through 4 demonstrated sequential agent coordination. The Location Loader provided data, the Geography Agent filtered it by proximity, and the Restaurant Agent refined it by dietary preferences. Each agent depended on the previous agent's output, creating a linear pipeline: load → filter → refine.

But not all agents have data dependencies. When planning a corporate outing, weather matters - but weather forecasts don't depend on restaurant data. The weather agent only needs to know the location name, which comes directly from your query. This independence enables a different execution pattern: parallel workflows.

The Context Agent Pattern

The weather agent represents a new agent archetype: the context provider. Unlike the Geography Agent (which transforms data by filtering) or the Restaurant Agent (which transforms data by applying domain logic), the weather agent provides decision-making context. It doesn't change the restaurant list or event list - it tells you whether outdoor activities make sense given current conditions.

Context providers have three defining characteristics:

They require only query parameters, not agent outputs. The weather agent needs a location name from your query, nothing more.

They return lightweight decision-making data, not bulk datasets. Weather forecasts contain perhaps 10-20 data points, not thousands of items.

They must reach synthesis unchanged. The synthesis model needs weather context to explain recommendations: "It's rainy, so I recommend indoor activities."

Parallel Execution

When the orchestrator analyzes your query "Find team-building activities in Seattle considering weather," it discovers that weather and activities can be fetched simultaneously. There's no dependency forcing sequential execution. The weather agent doesn't need activity data, and the activity agents don't need weather data.

The orchestrator generates a parallel workflow: fetch weather and activities in the same stage, then synthesize recommendations using both datasets. This is faster (parallel execution reduces total latency) and more natural (weather and activities are genuinely independent concerns).

The Context Agent Pattern

Enable the Weather Agent

Let's enable the Weather Agent now. In your bash terminal, execute:

```
./scripts/enable-agent.sh weather-agent
```

While You Wait

The enable process will start the pre-deployed Weather Agent ECS task. Once the task is healthy, it will register in CloudMap. The orchestrator will discover this agent alongside your existing agents when processing your next request. In the meantime, let's keep reading.

Data Dependencies Determine Execution Pattern

The Geography Agent must run after the Location Loader because it needs the restaurant list with coordinates. It can't filter by proximity without having locations to filter. This is a data dependency - one agent requires another agent's output.

The Restaurant Agent must run after the Geography Agent because it works better with pre-filtered data. While it could load all restaurants itself, receiving 465 nearby restaurants is more efficient than processing 1,819. This is a logical dependency - the agent can work standalone but benefits from running second.

The Weather Agent has no dependencies. It doesn't need restaurant lists, event lists, or filtered results. It needs the location name - "Seattle" - which comes directly from your query. There's no reason it must wait for other agents to complete.

What Weather Provides

The weather agent returns forecast information including:

Forecast array - Multi-day weather predictions with temperature, precipitation, wind, and conditions for each day.

Location confirmation - Verified location name to ensure the forecast matches your query.

Indoor recommendation - A boolean flag indicating whether conditions favor indoor activities. This is calculated based on precipitation probability, wind speed, and temperature extremes.

Average precipitation - The mean precipitation probability across forecast days, useful for understanding overall wetness.

This data is lightweight - perhaps 500-1000 tokens - but valuable for decision-making. When synthesis receives both weather context and activity options, it can filter or prioritize recommendations based on conditions: "It's rainy this weekend, so I recommend these indoor venues."

The Challenge: Keeping Context Alive

There's a technical challenge with context agents. In the sequential pipelines from Chapters 2-4, the orchestrator performed stage history pruning to save tokens. After the Geography Agent filtered restaurants, the original 1,819-restaurant dataset from the Location Loader was removed from memory. It wasn't needed anymore - future stages only needed the filtered results.

Stage history pruning checks: does any remaining stage reference this stage's output? If not, remove it to save tokens.

This creates a problem for context agents. The weather agent returns forecast data. No other agent needs forecasts - the Restaurant Agent doesn't check weather, the Events Agent doesn't check weather. Without special handling, the pruning logic would remove weather data after Stage 1 completes, before synthesis occurs.

The solution is the `wait_for` field. Setting `wait_for: true` on the weather agent tells the orchestrator: "Keep this agent's output for synthesis. Don't prune it even though no other agent references it." This ensures weather context survives until the synthesis model can use it to make weather-aware recommendations.

Testing Parallel Coordination

Making a Weather-Aware Request

Your Weather Agent is deployed and registered. The orchestrator now has four specialized agents available. Let's see how the Weather Agent changes recommendations by running the same query you tried at the end of Chapter 4.

Open Claude Code and clear your session first:

```
/clear
```

Run the same query you tried before deploying the Weather Agent:

```
let's use our mcp to find some fun activities in Seattle for the next few days
```

In Chapter 4, this returned a mix of indoor and outdoor activities with no weather context. Now that the Weather Agent is deployed, the orchestrator will fetch weather data in parallel and use it to inform recommendations.

What You'll See

Claude Code will discover four agents, generate a sophisticated conditional workflow, and return weather-aware activity recommendations. Here's what the output looks like:

Based on the weather forecast analysis from both agents, I recommend focusing on indoor team-building activities for this weekend in Seattle. The weather data shows heavy rain on Saturday (95% precipitation chance) and continued rain on Sunday (80% chance), making indoor venues the clear choice for successful team bonding.

Top Indoor Team-Building Recommendations:

Interactive & Social Activities:

- Karaoke venues like Novabox Karaoke or Seattle's Best Karaoke (\$25)
Perfect for team bonding and breaking down barriers through shared fun
- Comedy shows at Emerald City Comedy Club (\$50)
Shared laughter builds strong team connections
- Bowling or arcade venues
Classic team activities that encourage friendly competition

Cultural & Entertainment Options:

- Group movie experience at SIFF Cinema Uptown or AMC Pacific Place 11 (\$15)
Budget-friendly option for larger teams
- Live performances at 5th Avenue Theater or ACT Theatre (\$50)
Creates memorable shared experiences

Practical Considerations:

- Accessibility: Several venues offer wheelchair accessibility (Dimitriou's Jazz Alley, Benaroya Hall)
- Budget Flexibility: Options range from \$15 (cinemas) to \$50 (theaters)

Weekend Strategy:

Plan indoor activities for Saturday and Sunday due to heavy rain. However, Monday shows partly cloudy conditions (30% precipitation) - if your schedule allows, outdoor options could work then.

The orchestrator discovered four agents, generated a map-reduce workflow with parallel execution and aggregation, and completed successfully.

What Just Happened

This response demonstrates parallel agent coordination with weather context integration:

Discovery found four agents: When the orchestrator queried CloudMap, it found location-loader, geography-agent, restaurant-agent, and weather-agent in 0.3 seconds. It fetched all four agent cards and analyzed their capabilities, input schemas, and relationships.

Map-reduce workflow generated: The orchestrator analyzed your query and generated a map-reduce workflow that fetches weather and activity data in parallel, then aggregates results for synthesis.

Parallel execution with aggregation: Stage 1 executed four agents in parallel: weather-agent, location-loader (entertainment category), location-loader (events category), and location-loader (outdoor category). Stage 2 used a Joiner node to merge all results into a single aggregate output, which the orchestrator then synthesized into natural language recommendations.

Weather-aware recommendations: The synthesis model received merged data containing weather forecasts (95% precipitation Saturday, 80% Sunday) and 118+ venue options across categories. It prioritized indoor activities and explained why: "Heavy rain makes indoor venues the clear choice for successful team bonding."

The Map-Reduce Workflow Pattern

The workflow that executed behind the scenes followed this pattern:

Stage 1 - Map Phase (Parallel Data Gathering):

All four agent calls execute simultaneously in parallel:

- Weather Agent: Fetches 3-day forecast for Seattle (Heavy Rain 95% Sat, Rain 80% Sun, Partly Cloudy 30% Mon)
- Location Loader (entertainment): Loads 118 entertainment venues (theaters, karaoke, comedy clubs)
- Location Loader (events): Loads event venues
- Location Loader (outdoor): Loads 1,470 outdoor venues (parks, trails)

All four agents execute simultaneously. The orchestrator recognizes they have no dependencies on each other - all need only the location name "Seattle" from the query. Total Stage 1 time is dominated by the slowest loader, not the sum of all four.

Stage 2 - Reduce Phase (Aggregation & Synthesis):

- Joiner node waits for all four parallel calls to complete
- Merges all outputs into single `aggregate` data structure

- Prunes individual agent outputs to save tokens (no longer needed after merge)
- Orchestrator's GenericSynthesizer receives the aggregate
- Generates weather-aware recommendations in natural language (~14,000 tokens input)

Understanding wait_for in Parallel Workflows

The weather agent's output reaches synthesis because its capability has `wait_for: true`. Without this, stage history pruning would remove the weather data after Stage 1 completes.

Here's why pruning would happen: the Restaurant Agent doesn't reference weather data in its `input_mapping`. The Events Agent doesn't reference weather. No Stage 2 agent consumes weather output. The pruning logic concludes: "Weather data isn't needed by remaining stages, remove it to save tokens."

With `wait_for: true`, the orchestrator treats weather as synthesis-bound. The data isn't for intermediate agents - it's for the final synthesis step. This prevents pruning and ensures synthesis receives weather context alongside venue options.

Comparing to Chapter 4

The evolution from Chapter 4 to Chapter 5 introduces conditional logic and branching:

Chapter 4 (Three sequential agents): Location Loader → Geography → Restaurant → Synthesis. Linear pipeline, 14 vegan restaurants found, 3,181 tokens to synthesis, deterministic filtering.

Chapter 5 (Conditional with parallel gathering): (Weather || Entertainment Loader || Outdoor Loader) → Conditional Evaluation → Branch Synthesis. Parallel data gathering (118 + 1,470 venues), weather-dependent routing, 1,087 tokens to final synthesis, intelligent branching.

The sophistication increased dramatically. Chapter 4 filtered data through a linear pipeline. Chapter 5 gathered data in parallel, evaluated environmental conditions, and routed to appropriate synthesis strategies. The orchestrator didn't just run agents in parallel - it created branching logic based on weather forecasts.

How Parallel Workflows Work

Looking at the Orchestrator Logs

While your weather-aware activity request was being processed, the orchestrator was writing detailed logs about its parallel workflow generation decisions.

To view these logs, refer to the [Access CloudWatch Logs](#) guide and use the "Chapter 5 - CWLI" saved query to see all relevant component logs for this stage of the workshop.

Alternatively, you can view the specific log group at `/ecs/a2a-workshop/orchestrator` by navigating to CloudWatch > Log groups.

Here's what those logs showed during the workflow generation phase:

Agent Discovery Phase (0.3 seconds):

Discovered 4 agents: ['location-loader', 'weather-agent', 'geography-agent', 'restaurant-agent']
Retrieved agent cards from /.well-known/agent.json endpoints

Workflow Generation Phase (25.0 seconds):

Analyzing query: "Find team-building activities in Seattle for this weekend, considering weather"
Pattern selected: map-reduce
Stages required: 2
Nodes generated: 5

Node 0 - weather_check:

```
type: agent_call
agent_id: weather-agent
input_mapping: {
  days: "3",
  location: "Seattle"
}
```

Node 1 - load_entertainment:

```
type: agent_call
agent_id: location-loader
input_mapping: {
  category: "entertainment",
  location: "Seattle"
}
```

Node 2 - load_events:

```
type: agent_call
agent_id: location-loader
input_mapping: {
  category: "events",
  location: "Seattle"
}
```

Node 3 - load_outdoor:

```
type: agent_call
agent_id: location-loader
input_mapping: {
  category: "outdoor",
  location: "Seattle"
}
```

Node 4 - aggregate:

```
type: joiner
strategy: synthesize
wait_for: ["weather_check", "load_entertainment", "load_events", "load_outdoor"]
```

Execution Results:

Stage 1: 4 agents in parallel → weather (3-day forecast), entertainment (118 venues), events (2,590), outdoor (100 events)
Stage 2: Joiner aggregates all outputs → synthesis (134,503 tokens input) → final response

Total Duration: 42.2 seconds

The Workflow Structure: Map-Reduce Pattern

When the orchestrator detects multiple parallel data needs, it generates a `map-reduce` workflow pattern. This pattern has two stages:

Map Phase (Stage 1) - Multiple agents execute in parallel to gather data. Each agent produces independent output that will be aggregated.

Reduce Phase (Stage 2) - A Joiner node collects all parallel outputs, merges them into a single aggregate structure, then sends to synthesis for the final response.

Here's the workflow structure for your weather query:

Mermaid Diagram

Comparing Sequential vs Map-Reduce Workflow Structure

In Chapter 3's sequential workflow, the flow array looked like this:

```
{
  "pattern": "sequential",
  "flow": [
    ["load_restaurants"], // Stage 1
    ["filter_proximity"], // Stage 2 - waits for Stage 1
    ["synthesize"] // Stage 3 - waits for Stage 2
  ]
}
```

Each stage is a single-element array containing one agent ID. Execution is strictly linear: Stage 1 completes, then Stage 2 starts, then Stage 3 starts.

In Chapter 5's map-reduce workflow, the flow array shows parallel gathering and aggregation:

```
{
  "pattern": "map-reduce",
  "flow": [
    ["weather_check", "load_entertainment", "load_events", "load_outdoor"], // Stage 1 - parallel gathering
    ["aggregate"] // Stage 2 - aggregation & synthesis (reduce)
  ]
}
```

Stage 1 is a four-element array. All four agents execute in parallel. The orchestrator makes parallel API calls and waits for all four responses.

Stage 2 contains the Joiner node that aggregates all parallel outputs into a single structure for synthesis.

The wait_for Field in Map-Reduce Workflows

In map-reduce workflows, the `wait_for` field appears on the Joiner node, specifying which parallel outputs to aggregate:

```
{
  "id": "aggregate",
  "type": "joiner",
  "strategy": "synthesize",
  "wait_for": [
    "weather_check",
    "load_entertainment",
    "load_events",
    "load_outdoor"
  ]
}
```

This tells the orchestrator: "The aggregate joiner needs all four parallel outputs before proceeding." The Joiner waits for all agents in the `wait_for` list to complete, then merges their outputs into a single data structure.

The merge process creates an aggregate object containing all agent outputs:

```
{
  "weather_check": {
    "location": "Seattle",
    "forecast": [...],
    "average_precipitation": 68.3,
    "indoor_recommended": true
  },
  "load_entertainment": {
    "locations": [...118 venues...],
    "total_count": 118
  },
  "load_events": {
    "locations": [...2597 venues...],
    "total_count": 2597
  },
  "load_outdoor": {
    "locations": [...1470 venues...],
    "total_count": 1470
  }
}
```

This aggregate structure (134,503 tokens) is then sent to synthesis, which generates the final weather-aware response.

The Intelligence in Map-Reduce Orchestration

What makes this system powerful is the orchestrator's ability to generate map-reduce workflows dynamically. You didn't configure parallel data gathering. You didn't write aggregation logic.

The orchestrator:

1. Discovered four agents by querying CloudMap
2. Fetched their agent cards to understand input schemas and capabilities
3. Analyzed your query to identify multiple parallel data needs
4. Determined all four agents have query-only requirements (no agent dependencies)
5. Generated a map-reduce workflow with parallel gathering + aggregation
6. Executed all four agents simultaneously in Stage 1
7. Used a Joiner to aggregate all outputs in Stage 2

8. Sent the merged aggregate (134,503 tokens) to synthesis for weather-aware recommendations

All of this emerged from query analysis, agent metadata, and the orchestrator's reasoning capabilities. This is LLM-powered orchestration: intelligent, dynamic coordination that creates parallel execution patterns and aggregates diverse data sources into coherent responses, without explicit workflow programming.

Chapter 6: Build a Hotel Recommender Agent

Completing Your Travel Planning System

You've deployed agents for locations, geography filtering, restaurants, and weather. Now you'll build a hotel recommender agent to complete your travel planning system.

The Hotel Recommendation Challenge

Your corporate offsite attendees need accommodations near event venues. The hotel agent must filter by both location (using geography-agent) and amenities like gyms, pools, breakfast, and air conditioning.

This is similar to the restaurant agent's filtering, but with hotel-specific criteria. You'll adapt the restaurant agent pattern for a new domain - learning how to reuse proven patterns while handling domain-specific differences.

What You'll Build

You'll create a Hotel Agent that:

- Filters hotels by amenities (gym, pool, breakfast, air_conditioning)
- Filters by price range and minimum rating
- Supports dual-mode operation (pre-filtered or self-load)
- Integrates with geography-agent for proximity filtering

The agent loads data from `data/mock/accommodations/locations.json` containing ~150 hotels in Seattle with coordinates, ratings, prices, and amenity metadata.

By the end of this chapter, your travel planning system will be complete with agents for dining, weather, and accommodations - all coordinated by the orchestrator.

Ready to Build

You've seen how domain specialists work with restaurants. Now let's build one for hotels and enhance your travel planning system.

Prompt and Implement

Creating the Hotel Agent

You'll create a hotel recommender agent by adapting the restaurant agent as a template. This approach reuses proven code - only ~100 lines need domain-specific changes.

Your Experience Will Vary

What you see in Claude Code depends on your prompts and what Claude discovers. Keep your prompts to Claude Code clear and direct. Generally, the more information you provide to Claude Code, the better your results will be. Claude can make mistakes - that's normal. You can use Claude Code to help you fix issues as they arise.

Prompt Claude Code

Open Claude Code and provide a prompt like this. Feel free to modify and experiment. We will be using a custom created Claude Code slash command to create this. This command is defined in our `.claude/` folder and will help us through the process.

```
/workshop:create-agent I want to create a hotel agent using the restaurant agent as a template. It sh
```

What Claude Will Do

Claude will:

1. Read the restaurant agent to understand the pattern
2. Examine hotel data structure
3. Create the hotel agent code

4. May create deployment files (or you'll need to ask)

You'll see Claude using Read tools:

```
Read(agents/restaurant_agent/main.py)
Read(data/mock/accommodations/locations.json)
```

Claude may also use Bash tools:

```
Bash(find . -name "*.json" | grep -E "(agent|config)" | head -10)
```

Eventually, it will begin to Update and Write:

```
Update(lib/config.py)
Write(static/cloudformation/parameters/hotel-agent.json)
```

Because we are mostly copying and modifying an existing template, Claude should get most of this correct. Claude may ask you for permission to edit a file. It is often correct to do this, but be sure you understand what Claude is trying to do. If it looks like it is making a mistake, interrupt it by pressing `esc` or rejecting a request. Then tell the correct process.

Claude may not get everything right on the first try. Don't panic. Ask Claude to help you with whatever problem you're having. Effective AI-assisted development is iterative. Be specific about what's wrong and Claude will help you fix it.

Understanding the Filtering Logic

The Key Difference: ANY-OF vs ALL-OF

We should build the hotel agent using a different filtering logic than the restaurant agent. This difference matters for how we match user requirements.

Restaurant Agent: ANY-OF Logic

When someone asks for "vegetarian or vegan restaurants", they want places with **either** option:

```
# ANY-OF: Match if ANY restriction is satisfied
for restriction in dietary_restrictions:
    restaurants = [r for r in restaurants
                    if r.get("dietary", {}).get(restriction) is True]
```

Result: A restaurant with vegetarian=True and vegan=False **matches** the query.

Hotel Agent: ALL-OF Logic

When someone asks for "hotels with gym and pool", they want places with **both** amenities:

```
# ALL-OF: Match only if ALL amenities are present
for amenity in amenities:
    hotels = [h for h in hotels
               if h.get("amenities", {}).get(amenity) is True]
```

Result: A hotel with gym=True and pool=False **does not match** the query.

Why This Makes Sense

The filtering logic matches the domain requirements:

Dining - "vegetarian or vegan" means **either** works (ANY-OF) **Hotels** - "gym and pool" means **both** required (ALL-OF)

This is deterministic, metadata-driven filtering - no LLM calls, just boolean logic on structured data.

Deploy the Hotel Agent

Deployment Options

Claude may deploy the agent for you automatically. If not, you can deploy it manually using the deployment script. Or have Claude do it using the scripts!

Manual Deployment

If Claude hasn't deployed the agent, run:

```
./scripts/deploy-agent.sh hotel-agent
```

Custom Agent Deployment

Unlike the pre-deployed agents you've been enabling, this is a **custom agent** you're building from scratch. Custom agents require the full deployment process using `deploy-agent.sh` which builds the Docker image and creates the CloudFormation stack. Once deployed, you can use `enable-agent.sh` and `disable-agent.sh` to control this agent just like the pre-deployed ones.

This script:

- Creates ECR repository and CloudWatch log group
- Builds and pushes Docker image
- Deploys to ECS Fargate
- Registers with CloudMap for service discovery

Verify Deployment

Check the agent is running:

```
aws ecs describe-services \
  --cluster a2a-workshop-cluster \
  --services a2a-hotel-agent \
  --query 'services[0].{Status:status,Running:runningCount}'
```

Expected:

```
{
  "Status": "ACTIVE",
  "Running": 1
}
```

Or have Claude check! Claude is capable of running these AWS CLI commands.

Deployment Complete

The hotel agent is running in ECS and registered with CloudMap. The orchestrator can now discover and use it in multi-agent workflows.

Test the Pipeline

Testing Multi-Agent Coordination

With the hotel agent deployed, test a query that requires three agents working together.

The Test Query

We've been using Claude Code to create our hotel agent, so let's reset Claude and try it out. Let's create a new terminal we can test in. If something is broken, we want to reuse the terminal we've been working on our code in. Create a new terminal and start Claude Code in that new terminal and then test it:

```
please recommend hotels within walking distance of Pike Place Market with a gym
```

This query requires:

- **Location context:** Pike Place Market landmark in Seattle
- **Proximity filtering:** Walking distance (~1.2 km)
- **Amenity filtering:** Hotels with gym
- **Result limiting:** Top 20 rated hotels

Verify in CloudWatch Logs

View logs in [CloudWatch Logs Insights](#) using the "Chapter 6 - CWLI" saved query.

Location Loader (`/ecs/a2a-workshop/location-loader`):

```
✓ LOCATION LOADER OUTPUT - Complete dataset loaded:
location=Seattle category=accommodations total_count=132
```

Geography Agent (`/ecs/a2a-workshop/geography-agent`):

```
✅ Dynamic landmark resolved: 'Pike Place Market'
(lat=47.609486, lon=-122.341673)
Radius: 1.2 km (walking distance)
Filtered to 52 locations within range
```

Hotel Agent (`/ecs/a2a-workshop/hotel-agent`):

```
🏨 DUAL-MODE: Received 52 pre-filtered locations
Applying amenity filter: gym
Filtered to 37 hotels with gym
Sorted by rating, returning top 20
```

Multi-Agent Coordination Working

Your hotel agent successfully coordinates with location-loader and geography-agent. The 3-agent pipeline processed 132 → 52 → 37 hotels, delivering 20 top-rated results.

What You Should See

Claude Code will call the workshop orchestrator and you'll see a response like:

- Based on my search, here are the best hotel recommendations within walking distance of Pike Place Market

Closest Options (2-3 minute walk):

- The Palladian Hotel - 0.18 miles away, 4.5 rating
- Four Seasons Seattle - 0.19 miles away, luxury amenities including gym, pool, and business center

Nearby Options (4-5 minute walk):

- Mayflower Park Hotel Seattle - 0.25 miles away, 4.4 rating, pet-friendly
- Hotel Seattle - 0.37 miles away, 4.5 rating

Additional Options:

- Courtyard by Marriott - includes gym and pool
- Hyatt Place - gym and pool facilities
- citizenM Seattle - modern hotel with gym, pet-friendly

All of these hotels are consistently priced at \$150 per night and include standard amenities like Wi-Fi, parking along with their fitness facilities. The average rating across all options is 4.3/5, so you should like these properties.

For the shortest walk to Pike Place Market, I'd recommend The Palladian Hotel or Four Seasons Seattle, both are on foot and have excellent gym facilities.

What's Happening in our Workflow

Here's an example data flow from your test query:

Mermaid Diagram

You should recognize this as a sequential workflow, just like we had when we were filtering restaurants for vegetarian options.

If This Didn't Work

Don't worry. You can try to capture logs from CloudWatch and give them to Claude Code and try again, or you can move on. Our system should be robust enough that a failure of one agent should not impact the others.

Chapter 7: Adding Intelligence with LLM Reasoning (Optional)

From Deterministic Rules to Intelligent Judgment

In Chapter 6, you built the Hotel Agent and enhanced your travel planning system. You've deployed five agents that work together: location-loader provides data, geography-agent filters by proximity, restaurant-agent refines by dietary preferences, weather-agent adds context, and hotel-agent filters by amenities. These agents are precise and deterministic - they check metadata, calculate distances, and apply boolean logic.

But deterministic filtering has limits. Your agents can answer questions like "Which restaurants are vegan?" (check `dietary.vegan == true`) or "Which hotels have gyms?" (check `amenities.gym == true`). These are factual checks with clear yes/no answers.

What happens when the question requires judgment? Consider this planning scenario: "Find dinner and entertainment for 25 people near Pike Place Market with a \$2,000 budget and vegetarian options." Your existing agents can find vegetarian restaurants, locate entertainment venues, and calculate whether combinations fit the budget. But which combination is the **best use** of the budget? Should you prioritize a high-quality dining experience and modest entertainment? Or good dining with upgraded entertainment? What trade-offs make sense?

These aren't boolean checks. They're strategic decisions that require understanding value, evaluating trade-offs, considering experience quality, and making recommendations based on subjective preferences. This is where deterministic agents reach their limit and reasoning agents become essential.

What You'll Learn

This chapter introduces the reasoning agent pattern - agents powered by large language models that make intelligent decisions rather than checking metadata. You'll see how the Budget Agent works differently from your existing agents:

Restaurant Agent (deterministic) checks `dietary.vegan` field → true/false → filter. It's precise, fast (200ms), and consistent. It applies rules written by engineers.

Budget Agent (reasoning) evaluates budget allocation options → considers trade-offs → recommends strategy. It's thoughtful, slower (11s), and adaptive. It applies judgment guided by prompts.

The key difference is decision-making capability. The Restaurant Agent determines if a restaurant *qualifies* based on attributes. The Budget Agent determines which qualified option is *optimal* based on context, constraints, and goals. One filters facts. The other makes recommendations.

This is the first time you'll deploy an agent powered by Claude (or any LLM). Previous agents used the `BaseAgent` framework with deterministic Python functions. The Budget Agent uses `AgenticBaseAgent` with the Strands framework, which enables LLM-powered conversation, tool use, and multi-turn reasoning.

The Reasoning Challenge

Filtering vs Reasoning

The agents you've deployed so far are filters. They apply deterministic rules to structured data:

Geography Agent: `distance(restaurant, landmark) < 2000` → boolean → filter

Restaurant Agent: `dietary.vegan == true AND rating >= 4.0` → boolean → filter

Weather Agent: `precipitation_chance > 60` → set `indoor_recommended = true` → context

These agents are fast (200ms), consistent, and precise. They check facts. They don't interpret, judge, or recommend. When you ask "Which restaurants are vegan?", there's a definitive answer based on metadata.

Budget planning is different. When you ask "How should I allocate \$2,000 for 25 people?", there's no single correct answer. There are better and worse strategies based on:

- Your priorities (dining vs entertainment vs activities)
- Group dynamics (business meeting vs celebration)
- Context (Seattle weather, landmark proximity, dietary needs)
- Practical constraints (reservation requirements, large group logistics)

This requires **reasoning**, not filtering. The agent needs to:

1. Understand the request context (corporate event for 25 people)
2. Evaluate options (multiple valid budget allocations exist)
3. Make strategic decisions (prioritize dining experience? spread budget evenly?)
4. Explain recommendations (why this allocation makes sense)
5. Provide actionable advice (specific restaurants, dollar amounts, logistics)

Introducing LLM-Powered Agents

This is where large language models (LLMs) become essential. The agents you've deployed use the `BaseAgent` framework with deterministic Python functions. The Budget Agent uses `AgenticBaseAgent` with the Strands framework, which integrates LLM capabilities directly into agent processing.

With Strands, the Budget Agent can:

- **Reason** - Think through trade-offs, evaluate alternatives, consider constraints
- **Generate natural language** - Explain recommendations conversationally with dollar amounts and rationale
- **Make strategic decisions** - Choose optimal budget allocations based on context and goals

The Budget Agent doesn't just return filtered JSON like `{"restaurants": [...]}`. It returns strategic advice like:

"I recommend allocating \$1,000-1,200 (\$40-48/person) for the meal at Masala of India, which has extensive vegetarian options and \$\$ pricing. This leaves \$800-1,000 for logistics: gratuity (18-20% for large groups typically \$200-240), potential private dining room fees, and appetizers or beverages beyond the base meal."

Notice the difference: The Restaurant Agent tells you **which** restaurants are vegetarian. The Budget Agent tells you **how** to spend your budget wisely given those vegetarian options.

Enable the Budget Agent

Let's enable the Budget Agent now. In your bash terminal, execute:

```
./scripts/enable-agent.sh budget-agent
```

The enable process will start the pre-deployed Budget Agent ECS task. Once the task is healthy, it will register in CloudMap. Let's keep reading while it starts.

Deploy the Budget Agent

What the Budget Agent Does

The Budget Agent provides strategic recommendations for budget allocation in group events. Given a budget constraint, group size, and filtered options from other agents (restaurants, entertainment, accommodations), it evaluates allocation strategies, considers trade-offs, and generates natural language recommendations with specific dollar amounts and logistics advice.

Unlike your deterministic agents that filter data, the Budget Agent **reasons**. It doesn't determine which options fit the budget (that's arithmetic). It determines how to spend the budget wisely (that's strategy). When you have multiple valid options, the Budget Agent evaluates trade-offs like:

- Dining quality vs entertainment variety
- Experience optimization vs budget buffer
- Group size logistics vs cost per person
- Practical considerations (gratuuity, reservations, timing)

The Budget Agent uses the Strands Agent framework, which integrates Claude (via Bedrock) to enable LLM-powered reasoning. It makes tool calls to analyze data, evaluates options using natural language understanding, and generates conversational recommendations rather than filtered JSON.

Agent Framework: AgenticBaseAgent

The Budget Agent uses a different base class than your previous agents:

Previous Agents (location-loader, geography, restaurant, weather):

```
from lib.agents.agent_interface import BaseAgent

class RestaurantAgent(BaseAgent):
    def _process_task_impl(self, task, user_input):
        # Deterministic filtering with Python logic
        filtered_restaurants = [r for r in restaurants if r['dietary']['vegan']]
        return filtered_restaurants
```

Budget Agent:

```
from lib.agents.agentic_base import AgenticBaseAgent

class BudgetPlanningAgent(AgenticBaseAgent):
    def _define_agent_tools(self):
        # Define tools the LLM can use
        @tool(name="calculate_budget_allocation")
        def calculate_budget_allocation(total_budget: float, num_people: int):
            return {"per_person": total_budget / num_people}
        return [calculate_budget_allocation]

    def _get_agent_system_prompt(self):
        # Prompt that guides LLM reasoning
        return "You are a budget planning specialist..."
```

The `AgenticBaseAgent` framework handles:

- Initializing the Bedrock model connection
- Setting up Strands Agent with tools and system prompts
- Processing LLM responses
- Extracting structured data from conversational output
- Managing token usage and latency

Your deterministic agents respond in 100-300ms. The Budget Agent takes 10-12 seconds because it's actually reasoning through the problem using Claude.

Agent Input Schema: Multi-Agent Dependencies

The Budget Agent's input schema reveals its role as a reasoning agent that depends on other agents' results:

```

{
  "type": "object",
  "properties": {
    "query": {
      "type": "string",
      "description": "Natural language query about budget planning"
    },
    "total_budget": {
      "type": "string",
      "description": "Total budget amount (e.g., '2000')"
    },
    "num_people": {
      "type": "string",
      "description": "Number of people in the group"
    },
    "num_days": {
      "type": "string",
      "description": "Number of days for the trip",
      "default": "1"
    },
    "geography": {
      "type": "string",
      "description": "Location/city name"
    },
    "preferences": {
      "type": "string",
      "description": "Additional preferences or context"
    }
  },
  "required": ["query", "total_budget", "num_people"]
}

```

Notice the difference from previous agents. The Budget Agent doesn't require pre-filtered data arrays in its schema (like `{"locations": [...]}` or `{"restaurants": [...]}`). Instead, it receives:

- Natural language query
- Budget constraints (amount, group size, days)
- Context parameters (geography, preferences)

The orchestrator will pass filtered restaurant/entertainment results to the Budget Agent through the execution context, but they're not declared as required schema fields. This allows the Budget Agent to receive results from whatever agents the orchestrator determines are relevant.

The Budget Agent's capability metadata declares it as a reasoning agent that operates on results from domain specialists:

```
{
  "name": "validate-budget",
  "type": "reasoning",
  "description": "Provides strategic budget allocation recommendations...",
  "wait_for": true
}
```

Agent Output: Conversational Recommendations

The Budget Agent returns structured data with natural language explanations:

```
{
  "budget_allocation": {
    "total_budget": "2000",
    "num_people": "25",
    "num_days": "1"
  },
  "recommendations": "I recommend allocating $1,000-1,200 ($40-48/person) for the meal...",
  "reasoning_provided": true
}
```

The `recommendations` field contains natural language generated by Claude. It might be 500-2000 characters of conversational advice, not just filtered data. This output gets incorporated into the final synthesis, where the synthesis model can present the Budget Agent's strategic recommendations to you.

Unlike deterministic agents that return compact JSON (maybe 200-500 bytes), the Budget Agent returns 1,500-2,000 bytes because it's providing explanatory reasoning, not just facts.

How the Orchestrator Will Use It

When you make a query about budget planning, the orchestrator will:

Discover six agents: location-loader, geography-agent, restaurant-agent, weather-agent, hotel-agent, and budget-agent.

Analyze their capabilities to understand relationships. The deterministic agents (location-loader through hotel-agent) form data processing pipelines. The budget-agent is a reasoning agent that operates on their filtered results.

Generate a workflow where deterministic agents run first (loading, filtering, refining), then the budget-agent receives their results and generates strategic recommendations.

Set `wait_for: true` on the budget-agent node to ensure its reasoning output (which might be 1,500-2,000 bytes of natural language) reaches synthesis and gets incorporated into the final response.

This happens automatically through workflow generation. The orchestrator's planning LLM recognizes "budget planning" requirements in your query, discovers the budget-agent capability, and includes it in the generated workflow.

Testing Budget Reasoning

Making a Budget-Constrained Request

Your Budget Agent is deployed and registered. The orchestrator now has six specialized agents available: location-loader (data provider), geography-agent (proximity filtering), restaurant-agent (dietary filtering), weather-agent (context provider), hotel-agent (accommodation filtering), and budget-agent (strategic reasoning). Let's test this expanded capability by making a request that requires budget planning with multiple constraints.

Open Claude Code and clear your session first:

```
/clear
```

Now ask a question that requires strategic budget allocation:

```
i need to plan a dinner for 25 people in seattle near pike place. we need to support vegetarian options
```

This query requires multiple capabilities: geographic filtering (near Pike Place), dietary filtering (vegetarian options), and strategic budget allocation (how to spend \$2000 for 25 people). The orchestrator will coordinate multiple agents, culminating with the Budget Agent providing strategic recommendations.

What You'll See

Claude Code will discover six agents, generate a four-stage workflow with a reasoning agent, and return strategic budget allocation recommendations. Here's what the output looks like:

- I'll help you plan a dinner for 25 people in Seattle near Pike Place with vegetarian options and a \$2000 budget. Let me use the MCP workshop orchestrator to coordinate this planning task.

- a2a-workshop - ask_workshop (MCP)

(question: "I need to plan a dinner for 25 people in Seattle near Pike Place Market. Requirements: vegetarian options must be available, budget is \$2000 total. Please help me find suitable restaurants, estimate costs, and create a comprehensive dinner plan including venue recommendations, menu options, and logistics.")

| Based on the comprehensive analysis from multiple specialists, I can provide you with a well-structured dinner plan for your 25-person group near Pike Place Market within your \$2,000 budget.

- Perfect! I've got a comprehensive dinner plan for your group. Here's what the MCP workshop orchestrator recommends:

Top Restaurant Recommendation: Masala of India

- Why it's perfect: Extensive vegetarian menu, \$ pricing tier, accommodates groups
- Hours: Daily 10:00 AM - 10:00 PM
- Action: Contact them at masalaofindia.com to arrange seating for 25 people

Budget Breakdown (\$2,000 total = \$80/person)

- Main meal: \$1,000-1,200 (\$40-48/person)
- Gratuity: \$200-240 (18-20% for large groups)
- Extras: \$560-800 (appetizers, beverages, potential private room fee)

Backup Options

1. Ha! - International cuisine, 4.5-star rating, open until 2:00 AM
2. Mashiko - Sushi with vegan/vegetarian options

Logistics Recommendations

- Timing: Arrive 6:00-7:00 PM to avoid peak crowds
- Pre-planning: Call ahead to confirm vegetarian options and group seating
- Weather: All venues are indoor (perfect for Seattle's rainy weather)

- Location: All close to Pike Place Market for easy access

Next Steps

1. Contact Masala of India first to check availability
2. Confirm vegetarian menu options when booking
3. Ask about group pricing or set menus for 25 people
4. Consider pre-ordering to ensure smooth service

The orchestrator discovered six agents, generated a multi-stage workflow that included budget reasoning, and returned strategic recommendations with specific dollar amounts, venue suggestions, and logistics advice. The entire process took about 30-35 seconds, with the Budget Agent contributing 10-12 seconds for LLM-powered strategic reasoning.

What Just Happened

This response demonstrates sophisticated multi-agent coordination with a reasoning agent providing strategic recommendations:

Discovery found six agents: When the orchestrator queried CloudMap, it found location-loader, geography-agent, restaurant-agent, weather-agent, hotel-agent, and budget-agent. It analyzed their capabilities to understand which agents could contribute to budget-constrained dining planning.

Orchestrator reasoning about agent selection: The orchestrator's planning LLM analyzed the query and made intelligent choices about which agents to use:

****Critical Agent Declaration Analysis:****

- budget-agent declares: "Use me AFTER restaurant-agent and hotel-agent have filtered options. I provide the best recommendations when I can see the filtered results from other agents."
- This means budget-agent should receive filtered restaurants, not work independently

****Agent Selection & Dependencies:****

- location-loader: Load restaurant data for Seattle ✓
- geography-agent: Filter restaurants by proximity to Pike Place Market ✓
- restaurant-agent: Refine by vegetarian dietary restrictions ✓
- budget-agent: Analyze budget with filtered restaurant options ✓
- hotel-agent: Not relevant (looking for restaurants, not hotels) ✗
- weather-agent: Not relevant (indoor dining planning) ✗

****Pattern Choice:****

Sequential pattern - budget-agent explicitly declares it should receive filtered results from other agents, creating a clear dependency chain.

Four-stage sequential workflow generated: The orchestrator generated a workflow that processes data through filtering stages before applying reasoning: (1) data loading, (2) geographic filtering near Pike Place, (3) restaurant refinement for vegetarian options, and (4) budget strategy analysis on filtered results.

Deterministic filtering before reasoning: The Geography Agent filtered restaurants to those near Pike Place Market. The Restaurant Agent refined to vegetarian options. These deterministic steps created a manageable dataset. The Budget Agent then received these filtered results and reasoned about optimal allocation strategy.

Strategic recommendations with natural language: The Budget Agent didn't just return numbers. It generated conversational advice: specific restaurant names (Masala of India), dollar amount ranges (\$40-48/person for meal), logistics considerations (gratuity for large groups, timing advice), and backup options. This is LLM-generated strategic thinking, not filtered data.

The Multi-Agent Workflow in Action

The workflow that executed behind the scenes followed this sequential pattern:

Stage 1 - Location Loader: Fetched complete Seattle restaurant dataset, returning restaurants with full metadata (cuisine, dietary options, ratings, price tiers).

Stage 2 - Geography Agent: Received restaurant data, filtered to restaurants near Pike Place Market using haversine distance calculations.

Stage 3 - Restaurant Agent: Received nearby restaurants, filtered to those with vegetarian options by checking `dietary.vegetarian` and `dietary.vegan` metadata fields.

Stage 4 - Budget Agent: Received filtered vegetarian restaurant results from previous stages.

Constructed prompt: "Plan a travel budget allocation for 25 people visiting Seattle for 1 days with a total budget of \$2000." Invoked Claude via Bedrock. Made tool calls to

`calculate_budget_allocation` and `get_restaurants_by_price_tier`. Generated strategic reasoning about optimal budget allocation. Returned recommendations with specific dollar amounts and logistics advice.

Synthesis: Combined filtered restaurant options and budget strategy recommendations from all four stages into the comprehensive dinner plan you saw, with specific venues, pricing breakdown, and actionable next steps.

Comparing Deterministic and Reasoning Agents

This query highlights the complementary nature of deterministic and reasoning agents:

Restaurant Agent (Deterministic): "Which restaurants have `dietary.vegetarian == true`?" - Answer: 14 restaurants with vegetarian options near Pike Place Market. Fast (200ms), precise, factual.

Budget Agent (Reasoning): "How should I spend \$2,000 for 25 people at these 14 vegetarian restaurants?" - Answer: Strategic allocation (\$1,000-1,200 for meal, \$200-240 for gratuity, \$560-800 for extras), specific venue recommendation (Masala of India), logistics advice (timing, reservations, group considerations). Slower (11s), thoughtful, strategic.

The Restaurant Agent filtered facts. The Budget Agent made strategic recommendations based on those facts. Neither agent could replace the other:

- Without deterministic filtering, the Budget Agent would receive 1,819 restaurants and struggle to evaluate all options efficiently.
- Without budget reasoning, you'd have 14 vegetarian restaurant options but no guidance on how to allocate your budget optimally among them.

What Makes This Different

The Budget Agent's contribution showcases reasoning capabilities that deterministic agents can't provide:

Subjective Value Assessment: "Extensive vegetarian menu" and "accommodates groups" aren't boolean fields in metadata. The agent interpreted restaurant characteristics and made a judgment about suitability.

Trade-off Evaluation: Deciding to allocate \$1,000-1,200 for the meal vs other allocation strategies involves evaluating the trade-off between dining quality and budget buffer. This is strategic thinking.

Experience-Based Recommendations: Suggesting 6:00-7:00 PM timing and pre-ordering for large groups comes from understanding real-world dining logistics, not checking data fields.

Natural Language Explanation: The output reads like advice from an event planning professional: "Why it's perfect," "Backup Options," "Logistics Recommendations." This conversational framing makes recommendations actionable.

How LLM Agents Work

Looking at the Budget Agent Logs

While your budget planning request was being processed, the Budget Agent was writing detailed logs about its LLM reasoning process.

To view these logs, refer to the [Access CloudWatch Logs](#) guide and use the "Chapter 7 - CWLI" saved query to see all relevant component logs for this stage of the workshop.

Alternatively, you can view the specific log group at `/ecs/a2a-workshop/budget-agent` by navigating to CloudWatch > Log groups. Use Live Tail for real-time monitoring of LLM reasoning.

Here's what those logs showed during the Budget Agent's processing:

🚩 BUDGET INPUT - task_id=5db58271-2c4a-463d-a8fb-58f3aee0760c

```

query: ""
geography: "Seattle"
total_budget: "2000"
num_people: "25"
num_days: "1"
preferences: "group dining for 25 people near Pike Place Market, vegetarian options required"

```

🤖 Sending query to LLM: Plan a travel budget allocation for 25 people visiting Seattle for 1 days wi

🔧 Tool #1: calculate_budget_allocation

```

Input: {"total_budget": 2000, "num_people": 25, "num_days": 1}
Output: {
  "per_person_per_day": 80.0,
  "recommended_allocation": {
    "meals": 40,
    "entertainment": 24,
    "attractions": 16
  }
}

```

🔧 Tool #2: get_restaurants_by_price_tier

```

Input: {"price_tier": "$$", "location": "Seattle"}
Output: {
  "restaurants": [...],
  "count": 14
}

```

✅ LLM response received: 1622 chars

💰 LLM Reasoning Generated:

"Unfortunately, there are no vegetarian-friendly restaurants in the \$\$ price tier near Pike Place Market that fit the exact criteria. However, I can recommend several excellent options:

Given the group size of 25 people and the requirement for vegetarian options near Pike Place Market, I recommend the following approach:

Option 1: Allocate \$1,000-1,200 (\$40-48 per person) for the meal at a vegetarian-friendly restaurant like Masala of India...

[1,622 total characters of strategic reasoning]"

 BUDGET OUTPUT - Completed successfully

- ✓ task_id=5db58271-2c4a-463d-a8fb-58f3aee0760c
- ✓ duration=11.0s
- ✓ status=completed
- ✓ response_size=1838 bytes

How LLM Agents Process Differently

The Budget Agent's processing flow reveals fundamental differences from deterministic agents. Let's compare:

Deterministic Agent (Restaurant) processing:

```
1. Receive input: {"locations": [...], "dietary_restrictions": ["vegan"]}
2. Filter: filtered = [loc for loc in locations if loc['dietary']['vegan']]
3. Return: {"filtered_locations": filtered}
Duration: 200ms
```

Reasoning Agent (Budget) processing:

```
1. Receive input: {"total_budget": "2000", "num_people": "25", ...}
2. Construct LLM prompt: "Plan a travel budget allocation for 25 people..."
3. Invoke Claude via Bedrock (8-10 seconds)
4. LLM decides to call tools:
   - calculate_budget_allocation(2000, 25, 1)
   - get_restaurants_by_price_tier("$$", "Seattle")
5. LLM evaluates tool results and generates reasoning (1-2 seconds)
6. Extract response text: 1,622 characters of strategic advice
7. Return: {"recommendations": "I recommend allocating...", ...}
Duration: 11.0 seconds
```

The Budget Agent doesn't just execute code - it reasons. The LLM analyzes the problem, decides which tools to call, interprets their results, evaluates trade-offs, and generates conversational recommendations.

The LLM Reasoning Process

Let's break down what happened inside the Budget Agent's 11-second processing time:

Seconds 0-1: Prompt Construction

```
System Prompt: "You are a budget planning specialist for group travel...  
You have access to tools: calculate_budget_allocation,  
get_restaurants_by_price_tier, analyze_spending_options..."
```

```
User Query: "Plan a travel budget allocation for 25 people visiting  
Seattle for 1 days with a total budget of $2000. They need vegetarian  
dining options near Pike Place Market."
```

Seconds 1-3: Initial LLM Reasoning

Claude analyzes the query and determines: - Need to understand per-person budget: 25 people, \$2000 total = \$80/person - Should call `calculate_budget_allocation` to get baseline recommendations - Need to find vegetarian restaurant options in appropriate price tier

Seconds 3-5: Tool Call #1

Claude calls: `calculate_budget_allocation(total_budget=2000, num_people=25, num_days=1)`

Returns:

```
{  
  "per_person_per_day": 80.0,  
  "recommended_allocation": {  
    "meals": 40,  
    "entertainment": 24,  
    "attractions": 16  
  }  
}
```

Claude interprets: "Baseline suggests \$40/person for meals. For 25 people, that's \$1,000 for dining."

Seconds 5-7: Tool Call #2

Claude decides: "Need to check what \$\$ tier restaurants are available for vegetarian groups."

Calls: `get_restaurants_by_price_tier(price_tier="$$", location="Seattle")`

Returns: List of 14 restaurants including Masala of India, Ha!, Mashiko

Seconds 7-10: Strategic Evaluation

Claude reasons through options: - Masala of India: Extensive vegetarian menu, \$ pricing, good for groups - Ha!: 4.5 stars, international cuisine, open late (flexibility) - Mashiko: Sushi with vegan options, unique experience

Evaluates budget allocation: - \$1,000-1,200 for meal (\$40-48/person) provides quality dining - \$200-240 for gratuity (18-20% for large groups is standard) - \$560-800 remaining for extras (appetizers, beverages, private room fee)

Makes strategic decision: Prioritize dining experience (Masala of India) because vegetarian requirements + large group size means finding a restaurant that accommodates both is critical.

Seconds 10-11: Response Generation

Claude generates 1,622 characters of conversational advice: - Specific restaurant recommendation with rationale - Detailed budget breakdown with dollar amounts - Backup options ranked by suitability - Logistics advice (timing, reservations, group considerations) - Next steps for implementation

Second 11: Response Return

Budget Agent extracts the LLM response text and structures it into A2A protocol format with metadata.

Tool Use: How LLMs Decide What to Call

The Strands framework enables LLMs to use tools intelligently. Here's how the Budget Agent's tools are defined:

```

@tool(
    name="calculate_budget_allocation",
    description="Calculate recommended budget allocation for group travel"
)
def calculate_budget_allocation(
    total_budget: float,
    num_people: int,
    num_days: int
) -> dict:
    per_person = total_budget / num_people / num_days
    return {
        "per_person_per_day": per_person,
        "recommended_allocation": {
            "meals": per_person * 0.50,
            "entertainment": per_person * 0.30,
            "attractions": per_person * 0.20
        }
    }

```

When Claude sees this tool definition, it understands: - **Name:** What the tool does - **Description:** When to use it - **Parameters:** What inputs it needs (with types) - **Return type:** What output it provides

Claude decides: "I have a budget question. There's a tool called `calculate_budget_allocation` that takes `total_budget`, `num_people`, and `num_days`. I should call this first to get baseline numbers."

This is intelligent tool use, not hardcoded function calls. Different queries might result in different tool

sequences: - Query A: "What's a good budget?" → calls `calculate_budget_allocation` only -

Query B: "Find cheap restaurants" → calls `get_restaurants_by_price_tier("$")` only - Query C:

"Plan dinner for 25 people with \$2000" → calls both tools in sequence

The Strands Agent Framework

The Budget Agent uses the Strands library, which provides the infrastructure for LLM-powered agents:

Strands Agent Initialization (from `agents/budget_agent/main.py`):

```

from strands import Agent as StrandsAgent
from strands.models import BedrockModel

# Initialize Bedrock model
self.bedrock_model = BedrockModel(
    model_id="us.anthropic.claude-sonnet-4-20250514-v1:0",
    region_name="us-west-2",
    max_tokens=8096,
)

# Create Strands Agent with tools and system prompt
self.strands_agent = StrandsAgent(
    model=self.bedrock_model,
    system_prompt=self.system_prompt,
    tools=self.tools,
)

```

Invoking the LLM:

```

response = self.strands_agent(query)
response_text = response.text if hasattr(response, "text") else str(response)

```

Strands handles: - Bedrock API authentication and invocation - Tool definition formatting for Claude - Conversation state management (multi-turn if needed) - Response parsing and extraction - Error handling for failed LLM calls

Performance Breakdown: Where the 11 Seconds Go

Budget Agent Performance Analysis:

LLM Inference Time: 8-10 seconds

- └ Initial reasoning: 2-3 seconds
- └ Tool call decision: 1 second
- └ Tool #1 execution: 0.5 seconds
- └ Tool #1 result analysis: 1 second
- └ Tool #2 execution: 0.5 seconds
- └ Tool #2 result analysis: 1 second
- └ Final response generation: 2-3 seconds

Framework Overhead: 1 second

- └ Prompt construction: 0.3 seconds
- └ Bedrock API call setup: 0.2 seconds
- └ Response parsing: 0.3 seconds
- └ Data structuring: 0.2 seconds

Tool Execution: 1-2 seconds

- └ calculate_budget_allocation: 0.5 seconds
- └ get_restaurants_by_price_tier: 0.5-1.5 seconds

Total: 10-12 seconds

Compare this to deterministic agents: - Restaurant Agent: 0.2 seconds (Python filtering) - Geography Agent: 0.3 seconds (haversine calculations) - Budget Agent: 11 seconds (LLM reasoning)

The latency comes from actual thinking. Claude is analyzing your constraints, evaluating options, considering trade-offs, and generating strategic advice - not just checking boolean fields.

Cost Analysis: LLM Reasoning vs Deterministic Filtering

Cost Comparison (per request):

Restaurant Agent (Deterministic):

Compute: 0.2 seconds on Fargate (\$0.00001)

Total: \$0.00001 per request

Budget Agent (LLM Reasoning):

Compute: 11 seconds on Fargate (\$0.00005)

LLM Tokens:

└─ Input tokens: ~500 tokens (\$0.0001)

└─ Output tokens: ~400 tokens (\$0.0005)

Total: \$0.00065 per request

Cost Factor: ~65x higher for LLM reasoning

This cost difference is expected and acceptable: - Deterministic agents check facts (cheap, fast, simple) - Reasoning agents make strategic decisions (more expensive, slower, complex) - Use deterministic agents for filtering, reasoning agents for recommendations - The value of strategic advice often justifies the cost premium

What Makes This Reasoning

The Budget Agent demonstrates four reasoning capabilities:

- 1. Contextual Understanding:** Interpreted "group dining for 25 people" to mean need for large-group accommodations, group pricing considerations, and reservation logistics.
- 2. Trade-off Evaluation:** Evaluated \$1,000 for premium dining vs \$800 for good dining + \$400 better entertainment. Chose the former based on vegetarian requirements making restaurant selection more constrained.
- 3. Experience-Based Knowledge:** Recommended 6:00-7:00 PM timing and pre-ordering for large groups based on real-world dining experience, not data fields.
- 4. Explanation Generation:** Didn't just return "Masala of India" - explained WHY (extensive vegetarian menu), provided alternatives (Ha!, Mashiko), and gave actionable next steps (call ahead, confirm options, ask about group pricing).

These capabilities come from Claude's training, not from programmer-defined rules. The agent can adapt to different scenarios, handle novel requests, and provide human-like strategic thinking.

What's Next

You've seen inside the Budget Agent's reasoning process - from LLM prompt construction through tool use, strategic evaluation, and response generation. You understand how the Strands framework enables LLM-powered agents and how reasoning differs fundamentally from deterministic filtering.

In the next section, you'll learn about the reasoning agent pattern more broadly. You'll understand when to use reasoning agents vs deterministic agents, how `AgenticBaseAgent` differs from `BaseAgent`, and how the Budget Agent serves as a foundation for future reasoning agents (just as Restaurant Agent served as a foundation for Hotel Agent).

Inside LLM Reasoning

The Budget Agent's 11-second latency isn't overhead - it's Claude actually thinking through your budget constraints, evaluating options, and generating strategic recommendations. This is intelligent reasoning, not rule execution.

The Reasoning Agent Pattern

The Results

Let's examine what the Budget Agent accomplished in your dinner planning request. When you asked for dinner planning with budget constraints, the system coordinated multiple agents with distinct capabilities:

Restaurant Agent (deterministic) filtered to 14 vegetarian restaurants near Pike Place Market by checking `dietary.vegetarian` and `dietary.vegan` metadata fields - precise, fast (200ms), factual.

Budget Agent (reasoning) received those 14 options and generated strategic recommendations: allocate \$1,000-1,200 for dining (\$40-48/person), reserve \$200-240 for gratuity (18-20% for large groups), maintain \$560-800 buffer for extras, prioritize Masala of India for extensive vegetarian menu - thoughtful, slower (11s), strategic.

The deterministic agents filtered facts. The reasoning agent made strategic recommendations based on those facts. This demonstrates the complementary nature of deterministic and reasoning patterns: one provides precision, the other provides judgment.

Why LLM-Powered Reasoning Matters

Consider what the Budget Agent actually provides that deterministic agents cannot:

Deterministic Budget Checking (hypothetical):

```
def check_budget_fit(restaurant, budget, num_people):  
    cost = parse_price(restaurant['price']) * num_people  
    return cost <= budget # Returns: True or False
```

This tells you which restaurants fit the budget. It's precise, fast, and reliable. But it doesn't answer: - Is this a good **value** for the price? - Should I prioritize dining quality or entertainment variety? - What about gratuity, reservations, group logistics? - Which specific restaurant should I choose and why?

LLM-Powered Budget Reasoning (actual):

The Budget Agent generated:

"I recommend allocating \$1,000-1,200 (\$40-48/person) for the meal at Masala of India, which has extensive vegetarian options and \$\$ pricing. This leaves \$800-1,000 for logistics: gratuity (18-20% for large groups typically \$200-240), potential private dining room fees, and appetizers or beverages beyond the base meal."

This isn't just calculation - it's strategic planning: - **Value assessment**: "Extensive vegetarian options" evaluates quality, not just price - **Experience optimization**: Prioritizes dining quality for constrained vegetarian requirements - **Practical considerations**: Factors in gratuity norms, private room fees, group logistics - **Actionable advice**: Specific dollar ranges, venue recommendation, next steps

You can't achieve this with boolean checks or arithmetic. This requires understanding context, evaluating trade-offs, and making strategic decisions - capabilities that come from LLM reasoning.

The Four Agent Patterns

Your system now demonstrates four distinct agent patterns, each with specific strengths:

Pattern	Framework	Decision Method	Speed	Use Case	Example
Enhancement	BaseAgent	Mathematical	100-300ms	Objective filtering	Geography: distance < 2km
Domain Specialist	BaseAgent	Metadata checks	100-300ms	Boolean criteria	Restaurant: vegan == true
Context Provider	BaseAgent	External data	500-1000ms	Independent info	Weather: forecast API
Reasoning	AgenticBaseAgent	LLM judgment	10-15s	Strategic decisions	Budget: value trade-offs

The first three patterns use deterministic logic - rules defined by engineers. The reasoning pattern uses LLM inference - judgment guided by prompts. Both are valuable:

Deterministic Agents (BaseAgent): - Filter facts based on clear criteria - Fast (milliseconds), consistent, inexpensive - Perfect for objective decisions: "Which items qualify?" - Examples: Geographic proximity, dietary restrictions, price ranges

Reasoning Agents (AgenticBaseAgent + Strands): - Make strategic recommendations based on context - Slower (seconds), adaptive, more expensive - Perfect for subjective decisions: "What should I choose?" - Examples: Budget allocation, itinerary planning, experience optimization

When to Use Reasoning Agents

Reasoning agents solve problems that deterministic agents cannot:

Use Reasoning Agents When: - Questions require judgment, not just facts ("What's the best allocation?" vs "What fits the budget?") - Trade-offs must be evaluated (dining quality vs entertainment variety) - Context understanding is critical (group size affects logistics, weather affects venue choice) - Natural language explanations add value (users want to understand "why" recommendations make sense) - Adaptive responses are beneficial (same question with different contexts needs different answers)

Use Deterministic Agents When: - Criteria are objective and well-defined (distance, price, ratings, dietary restrictions) - Speed matters (filtering 1,000s of items in milliseconds) - Consistency is critical (same input must produce same output) - Cost optimization is important (no LLM inference costs) - Results must be verifiable (every decision has a checkable reason)

Real-World Example:

Query: "Find dinner for 25 people near Pike Place, vegetarian, \$2000 budget"

Deterministic filtering (fast, precise): - Geography Agent: Filter to restaurants within 2km of Pike Place Market - Restaurant Agent: Filter to restaurants with `dietary.vegetarian == true` - Result: 14 vegetarian restaurants nearby (200ms total)

Reasoning (strategic, thoughtful): - Budget Agent: "Given these 14 options and your \$2000 budget, I recommend Masala of India (\$40-48/person for meal, \$200-240 gratuity, \$560-800 buffer). Here's why..." - Result: Strategic allocation with rationale (11s)

Together: Fast filtering reduces search space, then strategic reasoning optimizes the choice within that space.

AgenticBaseAgent vs BaseAgent

The Budget Agent uses a different base class than previous agents. This architectural difference enables LLM capabilities:

BaseAgent (location-loader, geography, restaurant, weather, hotel):

```
from lib.agents.agent_interface import BaseAgent

class RestaurantAgent(BaseAgent):
    def _process_task_impl(self, task, user_input):
        # Deterministic Python logic
        restaurants = user_input.get('locations', [])
        dietary = user_input.get('dietary_restrictions', [])

        filtered = [r for r in restaurants
                    if any(r['dietary'].get(d) for d in dietary)]

        return self._create_success_task(task, filtered)
```

This is pure Python logic. No LLM calls, no tools, no reasoning - just filtering based on rules.

AgenticBaseAgent (budget-agent):

```

from lib.agents.agentic_base import AgenticBaseAgent
from strands import Agent as StrandsAgent
from strands.tools import tool

class BudgetPlanningAgent(AgenticBaseAgent):
    def _define_agent_tools(self):
        @tool(name="calculate_budget_allocation")
        def calculate_budget_allocation(total_budget: float, ...):
            return {...} # Tool implementation
        return [calculate_budget_allocation]

    def _get_agent_system_prompt(self):
        return "You are a budget planning specialist..."

    def _process_agentic_task(self, task, user_input):
        query = self._construct_llm_query(user_input)
        response = self.strands_agent(query) # LLM invocation
        response_text = (response.text if hasattr(response, "text")
                        else str(response))
        return self._create_success_task(task, response_text)

```

This integrates Strands Agent library for LLM capabilities. The agent: - Defines tools the LLM can call
 - Provides system prompt to guide reasoning - Invokes Claude via Bedrock - Processes LLM responses with tool use - Generates conversational recommendations

`AgenticBaseAgent` handles the complexity of LLM integration - Bedrock authentication, conversation management, tool framework, response parsing. You define the tools and prompts; the framework handles the infrastructure.

Architecture Implications

Adding reasoning agents changes system architecture in important ways:

Latency: - Deterministic workflow: 15-20 seconds total (10-12s workflow generation, 3-4s synthesis, 0.5s agent processing) - With reasoning agent: 30-35 seconds total (adds 11s for LLM inference) - Impact: Users notice longer response time but accept it for strategic advice

Cost: - Deterministic agents: \$0.00001 per request (compute only) - Reasoning agents: \$0.00065 per request (compute + LLM tokens) - Impact: 65x higher cost, but still pennies per request

Token Usage: - Deterministic agents return: 200-1,000 bytes (compact JSON) - Reasoning agents return: 1,500-2,000 bytes (conversational text) - Impact: Synthesis receives more data, but it's strategic context rather than bulk filtering results

Consistency: - Deterministic agents: Identical outputs for identical inputs - Reasoning agents: Adaptive outputs based on LLM reasoning (may vary slightly) - Impact: Less predictable, but more contextually appropriate

These trade-offs are expected and acceptable. Reasoning agents cost more and take longer because they're doing something fundamentally different: strategic thinking rather than rule execution.

System Evolution: Seven Chapters of Progress

The journey from Chapter 1 to Chapter 7 demonstrates how multi-agent systems evolve from simple data retrieval to intelligent strategic reasoning:

Chapter 1 - Single orchestrator with no agents: - Capability: Basic A2A protocol workflow - Result: Framework established, no specialized agents

Chapter 2 - Single data agent (Location Loader): - Capability: Bulk data loading - Result: All 1,819 restaurants, truncation required

Chapter 3 - Enhancement agent added (Geography): - Capability: Mathematical filtering (distance calculations) - Result: 465 nearby restaurants, no truncation

Chapter 4 - Domain specialist added (Restaurant): - Capability: Metadata-driven filtering (dietary restrictions) - Result: 14 vegan restaurants, perfect precision

Chapter 5 - Context provider added (Weather): - Capability: External context (parallel data fetching) - Result: Weather-aware recommendations

Chapter 6 - Second domain specialist (Hotel): - Capability: Amenity filtering (reusing patterns) - Result: Complete travel planning system

Chapter 7 - Reasoning agent added (Budget): - Capability: LLM-powered strategic recommendations - Result: Budget allocation with rationale and trade-off analysis

Each chapter added a new capability dimension. The system didn't just get larger - it got smarter. It progressed from data retrieval → filtering → refinement → context awareness → strategic reasoning.

Foundation for Future Agents

The Budget Agent serves as a foundation for future reasoning agents, just as the Restaurant Agent served as a foundation for the Hotel Agent:

Restaurant Agent → Hotel Agent (deterministic pattern reuse): - Both filter by metadata (dietary restrictions vs amenities) - Both use dual-mode operation (pre-filtered or self-load) - Both apply deterministic checks (boolean fields) - Lesson: Domain specialist pattern is reusable across domains

Budget Agent → Itinerary Agent (reasoning pattern reuse): - Both use LLM-powered decision-making - Both evaluate trade-offs and make strategic recommendations - Both use AgenticBaseAgent + Strands - Both generate conversational explanations - Lesson: Reasoning agent pattern is reusable for strategic planning tasks

In a future workshop extension, you might build an Itinerary Agent that: - Receives hotel, restaurant, and activity options from domain specialists - Uses LLM reasoning to create day-by-day travel itineraries - Evaluates trade-offs: travel time vs variety, relaxation vs activity, cost vs experience - Generates natural language itineraries with timing, logistics, and reasoning - Uses the same AgenticBaseAgent framework and Strands integration

The Budget Agent demonstrates the pattern. The Itinerary Agent would adapt it for a new domain.

What You've Accomplished

By completing Chapter 7, you've deployed a complete multi-agent system with six specialized agents demonstrating four distinct patterns:

✅ **Enhancement agents** for objective filtering (Geography) ✅ **Domain specialists** for metadata-driven refinement (Restaurant, Hotel) ✅ **Context providers** for independent information (Weather) ✅ **Reasoning agents** for strategic decision-making (Budget)

Your system can now: - Load bulk data efficiently - Filter by objective criteria (proximity, attributes, metadata) - Provide environmental context (weather, conditions) - Make strategic recommendations (budget allocation, trade-offs) - Coordinate all capabilities automatically through orchestrator workflow generation

The orchestrator discovers all six agents, analyzes their capabilities, generates appropriate workflows (sequential, parallel, or hybrid), and synthesizes results into comprehensive recommendations that are both factually accurate (from deterministic filtering) and strategically sound (from LLM reasoning).

Looking Forward

The multi-agent patterns you've learned extend beyond this workshop:

Additional Reasoning Agents could provide: - Itinerary planning (day-by-day schedules with trade-off analysis) - Experience optimization (balancing variety, cost, convenience) - Risk assessment (weather impacts, capacity constraints, timing conflicts) - Personalization (adapting recommendations to group preferences)

Hybrid Agents could combine patterns: - Start with deterministic filtering for efficiency - Apply LLM reasoning for strategic selection - Use tools for calculations and data fetching - Generate both structured data and natural language explanations

Production Enhancements could add: - Caching for repeated LLM calls (same query patterns) - Streaming for real-time reasoning visibility - Multi-turn conversations for iterative planning - User preference learning for personalized recommendations

The patterns are established. The framework is proven. The orchestrator handles complexity automatically. You can extend this system in any direction that adds value to your users.

Strategic Reasoning Achieved

Your multi-agent system now combines deterministic precision with LLM-powered strategic thinking, providing both factually accurate filtering and intelligent recommendations for complex planning scenarios.

Chapter 8: Build an Itinerary Planning Agent (Optional)

Completing Your Multi-Agent System

You've deployed agents for locations, geography filtering, restaurants, weather, hotels, and budget planning. Now you'll build an itinerary planning agent that creates complete multi-day travel schedules.

The Itinerary Planning Challenge

Your corporate event attendees need a complete multi-day itinerary for their Seattle visit. Creating an optimal itinerary requires strategic decisions: Should morning activities be outdoor when weather is better? Should lunch be near morning venues to minimize travel? How should you balance active vs relaxing experiences?

This is similar to the budget agent's strategic reasoning, but focused on scheduling instead of finances. You'll adapt the budget agent pattern for itinerary planning.

What You'll Build

You'll create an Itinerary Agent that:

- Creates day-by-day travel schedules using LLM reasoning
- Optimizes geographic sequencing to minimize travel time
- Balances activity types (cultural, dining, outdoor, entertainment)
- Adapts to weather conditions and time constraints
- Provides natural language explanations with timing and rationale

The agent uses the **AgenticBaseAgent** framework with Strands, just like the budget agent. It receives filtered hotels, restaurants, activities, and weather from other agents, then applies strategic scheduling logic to create comprehensive itineraries.

By the end of this chapter, you'll have your second LLM-powered reasoning agent, and you'll see how reasoning patterns transfer across domains.

Ready to Build

You've seen how the budget agent makes strategic allocation recommendations. Now let's build an itinerary agent that makes strategic scheduling recommendations.

Prompt and Implement

Creating the Itinerary Agent

You'll create an itinerary planning agent by adapting the budget agent as a template. This approach reuses proven reasoning agent code - the AgenticBaseAgent framework, Strands integration, and LLM-powered decision making.

Your Experience Will Vary

What you see in Claude Code depends on your prompts and what Claude discovers. Keep your prompts to Claude Code clear and direct. Generally, the more information you provide to Claude Code, the better your results will be. Claude can make mistakes - that's normal. You can use Claude Code to help you fix issues as they arise.

Prompt Claude Code

Open Claude Code and provide a prompt like this. Feel free to modify and experiment.

```
/workshop:create-agent let's create an itinerary agent based on our budget agent. we want this agent
```

What Claude Will Do

Claude will:

1. Read the budget agent to understand the reasoning agent pattern
2. Copy and adapt the agent for itinerary planning

3. Replace budget tools with itinerary-specific tools
4. Update the system prompt for scheduling strategy
5. Create deployment files (Dockerfile, parameters)
6. Ask before deploying

You'll see Claude using Read tools:

```
Read(agents/budget_agent/main.py)
Read(services/budget-agent/Dockerfile)
Read(static/cloudformation/parameters/budget-agent.json)
```

Then copying and adapting:

```
Write(agents/itinerary_agent/main.py)
Write(services/itinerary-agent/Dockerfile)
Write(static/cloudformation/parameters/itinerary-agent.json)
```

Because we are copying and modifying an existing reasoning agent template, Claude should get most of this correct. Claude may ask you for permission to deploy. This is correct - confirm when ready.

Working Iteratively

Claude may not get everything right on the first try. Common issues:

Wrong class name in type hints?

"you still have BudgetPlanningAgent in the get_agent() function signature. please change all references to ItineraryPlanningAgent"

Deployment errors?

"the deployment failed with: [paste error]. can you fix it?"

Agent not creating itineraries?

"the agent returned an error instead of an itinerary. check the CloudWatch logs at /ecs/a2a-workshop/itinerary-agent"

Key Mindset

Effective AI-assisted development is iterative. Be specific about what's wrong, and Claude will help you fix it. This is normal - you're learning to work collaboratively with AI assistants.

Important: Reasoning Agent Pattern

This is a **reasoning agent** using `AgenticBaseAgent`, not a simple filtering agent using `BaseAgent`. The itinerary agent should:

- Use LLM reasoning to make scheduling decisions
- Return natural language itineraries with explanations
- Make strategic trade-offs (travel time vs variety, pacing vs coverage)
- NOT just filter JSON data

In the next section, you'll learn more about what makes reasoning agents different from simple agents.

Adapting Reasoning Patterns

Building on Chapter 7

In Chapter 7, you learned how reasoning agents work. The Budget Agent uses `AgenticBaseAgent` with LLM reasoning to make strategic financial decisions. Now you're applying that same pattern to a different domain: scheduling.

Budget Agent vs Itinerary Agent

Both agents use the same `AgenticBaseAgent` framework, but solve different strategic problems:

Aspect	Budget Agent	Itinerary Agent
Strategic Question	"How should we allocate money?"	"How should we schedule time?"
Tools	<code>calculate_budget_allocation</code>	<code>calculate_travel_time</code> , <code>optimize_schedule</code>
Input Data	Restaurants, entertainment, budget	Hotels, restaurants, activities, weather
Trade-offs	Dining quality vs entertainment variety	Travel time vs activity variety
Constraints	Total budget limit	Time windows, weather conditions
Output	Budget breakdown with financial rationale	Day-by-day schedule with timing reasoning

The Adaptation Process

When Claude Code adapted the budget agent for itinerary planning, it:

1. **Kept the framework:** Same AgenticBaseAgent + Strands integration
2. **Replaced the tools:** Financial calculation → Travel time calculation
3. **Updated the system prompt:** Financial strategy → Scheduling strategy
4. **Changed the domain logic:** Money allocation → Time allocation

The underlying reasoning pattern is identical - only the domain changes.

Example: Strategic Reasoning in Both Domains

Budget Agent reasoning:

"I recommend allocating \$1,000-1,200 for dining at Masala of India (extensive vegetarian menu, \$ \$ pricing). This prioritizes dining quality given your vegetarian requirement, leaving \$800-1,000 for gratuity and logistics."

Itinerary Agent reasoning:

"I recommend scheduling Pike Place Market in the morning when it's less crowded, followed by lunch at El Borracho (0.3 miles away). This minimizes travel time and positions you for afternoon indoor activities when rain is forecast."

Both make strategic recommendations with rationale. Both evaluate trade-offs. Both explain their reasoning. The pattern is the same.

Why This Matters

Reasoning agent patterns are **reusable across domains**. You don't need to rebuild the architecture each time - you adapt:

- System prompts (financial strategy → scheduling strategy)
- Tools (budget calculations → travel time calculations)
- Domain logic (money constraints → time constraints)

The AgenticBaseAgent + Strands framework handles the rest: LLM invocation, tool use, conversation management, and response formatting.

Key Insight

Once you understand one reasoning agent, you can build others by adapting the domain-specific components. The framework and reasoning patterns transfer.

What's Next

Now that you understand how itinerary agents adapt the budget agent pattern, you'll deploy your itinerary agent and test the complete multi-agent system.

Deploy the Agent

Deployment Options

Claude may deploy the agent for you automatically. If not, you can deploy it manually using the deployment script. Or have Claude do it using the scripts!

Manual Deployment

If Claude hasn't deployed the agent, run:

```
./scripts/deploy-agent.sh itinerary-agent
```

Custom Agent Deployment

Unlike the pre-deployed agents you've been enabling, this is a **custom agent** you're building from scratch. Custom agents require the full deployment process using `deploy-agent.sh` which builds the Docker image and creates the CloudFormation stack. Once deployed, you can use `enable-agent.sh` and `disable-agent.sh` to control this agent just like the pre-deployed ones.

This script:

- Creates ECR repository and CloudWatch log group
- Builds and pushes Docker image
- Deploys to ECS Fargate
- Registers with CloudMap for service discovery

Verify Deployment

Check the agent is running:

```
aws ecs describe-services \
  --cluster a2a-workshop-cluster \
  --services a2a-itinerary-agent \
  --query 'services[0].{Status:status,Running:runningCount}'
```

Expected:

```
{
  "Status": "ACTIVE",
  "Running": 1
}
```

Or have Claude check! Claude is capable of running these AWS CLI commands.

Deployment Complete

The itinerary agent is running in ECS and registered with CloudMap. The orchestrator can now discover and use it in multi-agent workflows.

Test Multi-Agent Coordination

Testing the Complete System

Your itinerary agent is deployed. The orchestrator now has multiple specialized agents working together: location-loader (data), geography-agent (proximity), restaurant-agent (dietary), weather-agent (context), hotel-agent (amenities), budget-agent (financial strategy), and itinerary-agent (scheduling strategy).

Make the Test Request

We've been using Claude Code to create our itinerary agent, so let's reset Claude and try it out. Let's create a new terminal we can test in. If something is broken, we want to reuse the terminal we've been working on our code in. Create a new terminal and start Claude Code in that new terminal and then test it:

```
create a 2-day Seattle itinerary for 8 people staying near Pike Place Market. We need vegetarian rest
```

This query requires coordination across multiple agents: - **Weather Agent:** Checks forecast for 2 days (determines indoor/outdoor balance) - **Location Loader:** Loads hotels, restaurants, activities - **Geography Agent:** Filters by proximity to Pike Place Market - **Restaurant Agent:** Filters for vegetarian options - **Hotel Agent:** Filters for gym amenity - **Budget Agent:** Validates \$3000 budget allocation strategy - **Itinerary Agent:** Creates day-by-day schedule with timing and reasoning

What You Should See

Claude Code will call the workshop orchestrator and you'll see a response like:

Based on the comprehensive data analysis, here's your perfect 2-day Seattle itinerary for 8 people w

Hotel: The Palladian Hotel

- Only 0.18 miles from Pike Place Market
- Has a gym as requested
- \$150/night (\$300 total for 2 nights)

Day 1: Cultural Exploration

Morning:

- Pike Place Market (covered, perfect for rain)
- Lunch at El Borracho (Mexican with vegetarian options)

Afternoon:

- Market Theater (\$50/person)
- Coffee break in Pike Place area

Evening:

- Dinner at Limoncello Belltown (Italian with vegetarian dishes)
- SIFF Cinema Downtown (\$15/person)

Day 2: Entertainment & Local Flavor

Morning:

- Boeing IMAX Theater (\$15/person)
- Lunch at Food Junction (Indian street food, vegetarian options)

Afternoon:

- Benaroya Hall - Seattle Symphony (\$50/person)
- Explore Seattle Center covered areas

Evening:

- Dinner at Menya Musashi Niten Ichiryu (ramen with vegan/vegetarian options)
- Dimitriou's Jazz Alley (\$50/person)

Budget Breakdown (\$3,000 total):

- Hotels: \$600
- Dining: ~\$1,200
- Entertainment: ~\$900
- Activities: ~\$300

The agents considered the weather forecast (heavy rain both days) and focused on indoor activities and

Understanding the Coordination

This demonstrates sophisticated multi-agent coordination:

Deterministic Filtering (fast, precise): - Geography: Proximity to Pike Place Market - Restaurant: Vegetarian metadata checks - Hotel: Gym amenity verification

Context Provision (independent): - Weather: 2-day forecast with indoor recommendations

Strategic Reasoning (LLM-powered): - Budget: Financial allocation analysis - Itinerary: Temporal scheduling with trade-off evaluation

The itinerary agent receives filtered results from multiple agents and generates strategic scheduling recommendations that optimize timing, minimize travel, adapt to weather, and stay within budget.

Verify in CloudWatch Logs

View logs in [CloudWatch Logs Insights](#) using the "Chapter 8 - CWLI" saved query.

Itinerary Agent (`/ecs/a2a-workshop/itinerary-agent`):

```
✔ ITINERARY PLANNING - Multi-day scheduling with strategic reasoning
Day 1: Cultural focus with covered venues (rain expected)
Day 2: Entertainment venues within walking distance
Total processing time: 18 seconds (LLM reasoning + tool calls)
```

Multi-Agent Coordination Working

Your system coordinates deterministic filtering, context provision, and strategic reasoning across multiple agents to generate comprehensive, optimized itineraries.

If This Didn't Work

Don't worry. You can try to capture logs from CloudWatch and give them to Claude Code and try again, or you can move on. Our system should be robust enough that a failure of one agent should not impact the others.

What You've Learned

By completing this chapter, you've:

- Built a reasoning agent by adapting an existing reasoning agent template
- Understood the difference between filtering agents (BaseAgent) and reasoning agents (AgenticBaseAgent)
- Seen how LLM-powered strategic decision making works with tools and system prompts
- Deployed and tested multi-agent coordination across 7+ specialized agents
- Created a complete travel planning system that handles deterministic filtering, context awareness, and strategic reasoning

The itinerary agent demonstrates how reasoning patterns transfer across domains. The same AgenticBaseAgent + Strands framework that powers budget planning also powers itinerary planning - only the tools, system prompt, and strategic logic change.

Epilogue: Your Path Forward

What You've Accomplished

You started this workshop with a single agent returning 1,819 unfiltered restaurants, requiring three truncation iterations to fit within token budgets. Random sampling eliminated 43% of your data. Results were generic and unfocused.

You end this workshop with a sophisticated multi-agent system that:

- Coordinates 7+ specialized agents automatically
- Filters data deterministically through multiple stages
- Adapts workflows based on query requirements
- Provides highly-relevant, budget-aware recommendations with strategic reasoning
- Operates within token budgets without truncation
- Handles conditional logic and parallel execution

You've moved from single-agent limitations to multi-agent coordination with both deterministic filtering and LLM-powered reasoning.

Your Multi-Agent Journey

The Complete Progression

Here's the evolution of your multi-agent system from Chapter 2 through Chapter 8:

Chapter	Agents	Pattern	Data Flow	Result Quality
Chapter 2	1 agent	Single query-response	1,819 items → synthesis	Random sampling 43% data loss
Chapter 3	2 agents	Sequential pipeline	1,819 → 465 → synthesis	Geographic filtering All nearby
Chapter 4	3 agents	Sequential refinement	1,819 → 465 → 14 → synthesis	Multi-criteria matching Precise results
Chapter 5	4 agents	Conditional + Parallel	Weather → Branch (Indoor/Outdoor)	Weather-aware routing Context-driven
Chapter 6	5 agents	Domain specialist	Hotels filtered by amenities	Amenity-based filtering Hotel recommendations
Chapter 7	6 agents	Reasoning agent (LLM)	Budget calculations with reasoning	Strategic recommendations Budget analysis
Chapter 8	7 agents	Reasoning agent (LLM)	Multi-day scheduling with reasoning	Strategic itineraries Temporal optimization

System Capabilities Comparison

Chapter 2 System Could:

- Load restaurant data for a location
- Return results (with significant truncation)

Your Final System Can:

- Load multiple data types (restaurants, entertainment, weather)
- Filter by geographic proximity (haversine calculations)
- Refine by domain criteria (dietary preferences, categories)
- Evaluate environmental conditions (weather forecasts)
- Branch execution based on conditions (indoor vs outdoor focus)
- Calculate cost combinations across multiple agents

- Filter by budget constraints
- Rank options by various criteria (budget fit, proximity, ratings)
- Provide context-aware, multi-criteria recommendations

Your system evolved from "here are some restaurants" to "here's a complete 2-day itinerary with budget-appropriate restaurant and entertainment combinations for your group size, filtered by dietary needs and location, adapted for weather conditions, with strategic scheduling and timing recommendations."

Journey Complete

From single-agent truncation to sophisticated multi-agent coordination, you've built a system that demonstrates the core principles of A2A-based inter-agent systems. Well done!

Cleanup

If you're doing this as part of an AWS event, no need to worry about this. We'll clean up everything for you. If you're doing this on your own, be sure to clean up any stacks to avoid costs.

Cleaning Up Workshop Resources

Delete Learner-Created Agent Stacks

Delete the custom agent stacks you created during the workshop:

```
./scripts/delete-agents.sh
```

Workshop Cleanup

The `delete-agents.sh` script only deletes learner-created agents (hotel-agent and itinerary-agent). The pre-deployed agents (location-loader, geography-agent, restaurant-agent, weather-agent, budget-agent, events-agent) are part of the main CloudFormation stack and will be deleted when you delete the entire workshop infrastructure. To delete everything, remove the main `a2a-workshop` CloudFormation stack from the AWS Console (see below).

This deletes only the custom agents you deployed: - hotel-agent (Chapter 6) - itinerary-agent (Chapter 8)

The pre-deployed agents, workshop infrastructure (Code Server, MCP Gateway, Orchestrator, VPC), and all other resources remain active.

Delete Entire Workshop (Self-Paced Only)

If you deployed this workshop yourself (not via AWS Event), delete the main CloudFormation stack:

1. Go to [AWS CloudFormation Console](#)
2. Select the `a2a-workshop` stack
3. Click **Delete**
4. Confirm deletion

This deletes all workshop infrastructure: Code Server, MCP Gateway, Orchestrator, VPC, ECS cluster, Application Load Balancer, and all nested stacks.

Take the Workshop Home

Download the Complete Workshop Guide

Don't forget to download the comprehensive PDF guide! It includes all chapters, diagrams, code examples, and design patterns in a single offline-friendly document. ****Download:**** [workshop-guide.pdf](https://ws-assets-prod-iad-r-pdx-f3b3f9f1a7d6a3d0.s3.us-west-2.amazonaws.com/c079209e-3f27-4614-9b4f-48eae3921e57/workshop-guide.pdf) (~5MB, 100+ pages) This is perfect for: - Reviewing concepts after the workshop - Sharing with colleagues - Reference during your own multi-agent implementations - Offline access to all workshop content

Final Thoughts

From a single agent with truncation problems to a sophisticated multi-agent system with reasoning capabilities—you've come a long way.

The principles you've learned—separation of concerns, progressive refinement, deterministic filtering, intelligent orchestration—apply far beyond restaurant recommendations. They're fundamental to building reliable, scalable AI systems.

Keep experimenting. Keep building. Keep learning.

Workshop Complete

You've completed the Inter-Agent Systems workshop. You've learned A2A protocol fundamentals, deployed production agents, built custom logic, and mastered multi-agent coordination patterns. The foundation is yours. Where you go from here is up to you. Thank you for your time and effort. Build something amazing.